TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN VẠN VẬT KẾT NỐI

BÁO CÁO CUỐI KỲ

THIẾT KẾ HỆ THỐNG ĐIỂM DANH SINH VIÊN TỰ ĐỘNG SỬ DỤNG RFID

Giảng viên hướng dẫn: ThS. Nguyễn Thanh Nhã

Sinh viên thực hiện:

Nguyễn Đức Học	24162039
Phan Khánh An	24162004
Lê Đặng Hoàng Anh	24162006
Trần Công Khánh	24162061
Nguyễn Bá Nam	24162077

TP. HCM, ngày 1 tháng 4 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm: ……..

KÝ TÊN

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành đến quý thầy cô Khoa Công nghệ Thông tin – Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh đã tạo điều kiện tốt nhất để em được tiếp cận kiến thức thực tế và áp dụng các kiến thức chuyên ngành đã học vào đề tài thiết kế hệ thống IoT trong môi trường học thuật chuyên nghiệp Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến ThS. Nguyễn Thanh Nhã, người đã trực tiếp hướng dẫn và truyền đạt những kiến thức nền tảng quý báu cho em trong suốt quá trình thực hiện đề tài. Những lời góp ý, định hướng chuyên môn và sự tận tình của Thầy đã giúp bài báo cáo giữa kỳ của em được hoàn thiện và đạt được kết quả như mong muốn. Bài báo cáo là kết quả đúc kết từ những bài học trong quá trình nghiên cứu, xây dựng Firmware và phát triển Backend của em. Tuy nhiên, do hạn chế về mặt kiến thức chuyên môn và thời gian thực hiện nên không thể tránh khỏi những thiếu sót. Em rất mong có được những ý kiến đóng góp của Thầy để bài báo cáo cũng như hệ thống được hoàn thiện hơn trong tương lai. Để có thể hoàn thành được đề tài này, em đã nhận được sự quan tâm và giúp đỡ của nhiều cá nhân, tập thể trong và ngoài trường. Một lần nữa em xin bày tỏ lòng biết ơn sâu sắc về sự hỗ trợ quý báu này.

TP. HCM, ngày 1 tháng 4 năm 2026

Sinh viên

Nhóm 9

MỤC LỤC

LỜI CẢM ƠN II
DANH MỤC CÁC TỪ VIẾT TẮT VII
DANH SÁCH HÌNH ẢNH IX
DANH SÁCH BẢNG X
CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI 1
1.1. Đặt vấn đề 1
1.2. Tình hình nghiên cứu 1
1.2.1. Các hệ thống trong nước 1
1.2.2. Các hệ thống nước ngoài 2
1.3. Lý do chọn đề tài 2
1.4. Mục tiêu và phạm vi đề tài 3
1.4.1. Mục tiêu đề tài 3
1.4.2. Phạm vi đề tài 3
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT 4
2.1. Tổng quan về vi điều khiển ESP32 4
2.1.1. Kiến trúc và đặc tả kỹ thuật 4
2.1.2. Hệ điều hành thời gian thực (RTOS) 5
2.2. Công nghệ nhận dạng vô tuyến RFID 6
2.2.1. Khái niệm và nguyên lý hoạt động 6
2.2.2. Đặc tính tần số và Module MFRC522 6
2.2.3. Cấu trúc bộ nhớ thẻ Mifare Classic 1K 8
2.2.4. Giao thức ngoại vi SPI và I2C 8
2.2.5. Giao thức HTTP và Kiến trúc RESTful API 8
2.2.6. Định dạng dữ liệu JSON 8
2.3. Công nghệ Web và cơ sở dữ liệu 9
2.3.1. Hệ quản trị CSDL quan hệ (RDBMS) và tính chất ACID 9
2.3.2. Khung phần mềm Web (Web Framework) 9

2.3.3. Xung đột luồng (Race Condition) và Đồng bộ hóa (Synchronization) 10
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG 11
3.1. Phân tích yêu cầu và Sơ đồ khối kiến trúc hệ thống 11
3.1.1. Phân tích yêu cầu hệ thống 11
3.1.2. Sơ đồ khối tổng quát 11
3.1.3. Kiến trúc Bộ đệm thông minh 12
3.2. Thiết kế Cơ sở dữ liệu 12
3.2.1. Sơ đồ quan hệ thực thể (ERD) 13
3.2.2. Đặc tả chi tiết các bảng dữ liệu 13
3.3. Thiết kế Giải thuật và Logic điều khiển 18
3.3.1. Giải thuật xử lý tại thiết bị biên (Firmware ESP32) 18
3.3.2. Giải thuật đồng bộ và xử lý logic tại máy chủ Backend 19
3.4. Thiết kế Giao tiếp API 19
3.5. Thiết kế Giao diện Web Quản trị (Frontend) 21
3.5.1. Kiến trúc trang đơn 21
3.5.2. Thiết kế luồng tương tác người dùng (UI/UX) 21
3.6. Lưu đồ thuật toán 21
3.6.1. Lưu đồ xử lý chính của ESP32 (Core 1 - Quét thẻ & Hiển thị) 22
3.6.2. Lưu đồ truyền tải mạng ngầm ESP32 (Core 0 - TaskUpload) 23
3.6.3. Lưu đồ thuật toán xử lý điểm danh trên Server (FastAPI) 24
3.6.4. Lưu đồ xác thực đăng nhập Giao diện Web (Frontend) 25
CHƯƠNG 4 XÂY DỰNG VÀ THIẾT LẬP MÔ PHỎNG 26
4.1. Môi trường phát triển 26
4.2. Xây dựng Firmware(Wokwi) cho ESP32 26
4.2.1. Cấu trúc điều khiển đa nhiệm (FreeRTOS) 27
4.2.2. Cơ chế Local Caching và đồng bộ hóa API 27
4.2.3. Xử lý logic nhận dạng và phản hồi người dùng 28
4.2.4. Đồng bộ thời gian thực qua NTP 28

4.3. Xây dựng Backend 29
4.3.1. Tổng quan kiến trúc Backend 29
4.3.2. Thiết kế cơ sở dữ liệu (Database Schema) 30
4.3.3. Xây dựng các API giao tiếp với ESP32 30
4.3.4. Xây dựng các API phục vụ quản lý (Web Dashboard) 31
4.4. Xây dựng giao diện Web 31
4.4.1. Lựa chọn công nghệ và Kiến trúc Frontend 32
4.4.2. Xây dựng trang Xác thực người dùng 32
4.4.3. Xây dựng Bảng điều khiển (Dashboard) và Quản lý lớp học 34
4.4.4. Tối ưu hóa Trải nghiệm người dùng (Advanced UI/UX) 35
4.4.5. Cơ chế tương tác API và Cập nhật dữ liệu thời gian thực 36
4.5. Thiết lập thông số mô phỏng 37
4.6. Thiết kế các kịch bản mô phỏng 37
CHƯƠNG 5 THIẾT LẬP VÀ TRIỂN KHAI PHẦN CỨNG 38
5.1. Chuẩn bị linh kiện và thiết lập môi trường phát triển 38
5.2. Lắp ráp và kiểm thử theo từng phân hệ (Incremental Testing) 39
5.3. Tích hợp hệ thống và xử lý sự cố vận hành thực tế 39
5.3.1. Xử lý lỗi đồng bộ thời gian (Lỗi 400 Bad Request) 40
5.3.2. Xử lý lỗi vi phạm toàn vẹn dữ liệu (Lỗi 500 Internal Server Error) 41
CHƯƠNG 6 KẾT QUẢ, ĐÁNH GIÁ MÔ PHỎNG VÀ NGOÀI THỰC NGHIỆM ... 42
6.1. Kết quả, đánh giá mô phỏng 42
6.1.1. Kịch bản 1: Đánh giá tốc độ phản hồi của hệ thống (Latency) 42
6.1.2. Kịch bản 2: Đánh giá độ ổn định khi chịu tải cao 43
6.1.3. Kịch bản 3: Đánh hệ thống trong trường hợp ngoại tuyến 43
6.2. Kết quả, đánh giá ngoài thực nghiệm 44
6.2.1. Kịch bản 1: Đánh giá tốc độ phản hồi của hệ thống (Latency) 44
6.2.2. Kịch bản 2: Đánh giá độ ổn định khi chịu tải cao 45
6.2.3. Kịch bản 3: Đánh giá độ ổn định của hệ thống khi vận hành khi ngoại tuyến 46

CHƯƠNG 7 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 47
TÀI LIỆU THAM KHẢO .....
PHỤ LỤC .....

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ Tiếng Anh đầy đủ	Giải nghĩa Tiếng Việt / Chú thích
3NF	Third Normal Form	Dạng chuẩn hóa 3 (trong thiết kế Cơ sở dữ liệu)
AC	Alternating Current	Dòng điện xoay chiều
ACID	Atomicity, Consistency, Isolation, Durability	4 tính chất của giao dịch CSDL: Tính nguyên tử, Tính nhất quán, Tính cô lập, Tính bền vững
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tín hiệu tương tự sang số
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
ASGI	Asynchronous Server Gateway Interface	Giao diện cổng máy chủ bất đồng bộ
BaaS	Backend-as-a-Service	Dịch vụ máy chủ cung cấp sẵn
BLE	Bluetooth Low Energy	Bluetooth tiêu thụ năng lượng thấp
CPU	Central Processing Unit	Bộ vi xử lý trung tâm
CSDL	Database	Cơ sở dữ liệu
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản trên dữ liệu (Tạo, Đọc, Cập nhật, Xóa).
DAC	Digital-to-Analog Converter	Bộ chuyển đổi tín hiệu số sang tương tự
DB	Database	Cơ sở dữ liệu
DC	Direct Current	Dòng điện một chiều
DMIPS	Dhrystone Million Instructions Per Second	Triệu lệnh Dhrystone trên giây (Đơn vị đo hiệu suất)

DOM	Document Object Model	Mô hình đối tượng tài liệu
EEPROM	Electrically Erasable Programmable Read-Only Memory	Bộ nhớ đọc chỉ dữ liệu có thể xóa bằng điện (Cấu trúc thẻ Mifare).
ERD	Entity-Relationship Diagram	Sơ đồ quan hệ thực thể.
GPIO	General-Purpose Input/Output	Chân tín hiệu vào/ra đa dụng.
HF	High Frequency	Tần số cao.
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản.
I2C	Inter-Integrated Circuit	Giao thức truyền thông nối tiếp đồng bộ mạch tích hợp.
IoT	Internet of Things	Vạn vật kết nối.
I/O	Input/Output	Luồng nhập/xuất dữ liệu (liên quan đến các thao tác đọc ghi).
ISO	International Organization for Standardization	Tổ chức tiêu chuẩn hóa quốc tế (Dùng cho chuẩn ISO/IEC 14443).
JS	JavaScript	Ngôn ngữ lập trình kịch bản thường dùng cho ứng dụng Web.
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản nhẹ.
JWT	JSON Web Token	Chuỗi mã thông báo bảo mật Web.
LF	Low Frequency	Tần số thấp.
MISO	Master In Slave Out	Đường truyền dữ liệu từ thiết bị tớ về thiết bị chủ.
MOSI	Master Out Slave In	Đường truyền dữ liệu từ thiết bị chủ đến thiết bị tới.
MSSV	Student ID	Mã số sinh viên.
Mutex	Mutual Exclusion	Khóa loại trừ tương hỗ.
NFC	Near-Field Communication	Công nghệ giao tiếp tầm ngắn.

DANH SÁCH HÌNH ẢNH

Hình 2.1. Sơ đồ chân của bo 30 chân ESP32 DevKit - Ảnh: ELECTRONICS HUB [4]....4
Hình 2.2. Hình ảnh minh họa một dự án dùng FreeRTOS 5
Hình 2.3. Nguyên lý hoạt động của RFID - Ảnh: Trackify....6
Hình 2.4. Module MFRC522....7
Hình 2.5. Hình ảnh minh họa giao thức SPI....8
Hình 2.6. Minh họa Race Condition - Ảnh GeeksforGeeks....10
Hình 3.1. Sơ đồ khối tổng quát của hệ thống 12
Hình 3.2. Sơ đồ quan hệ thực thể (ERD)....13
Hình 3.3. Lưu đồ thuật toán xử lý nhận dạng và hiển thị trên Core 1 của vi điều khiển ESP32 22
Hình 3.4. Lưu đồ thuật toán tác vụ truyền tải mạng ngầm (TaskUpload) trên Core 0....23
Hình 3.5. Lưu đồ thuật toán xử lý điểm danh và cơ chế đồng bộ luồng (Mutex Lock) trên máy chủ Backend....24
Hình 3.6. Lưu đồ luồng tương tác và xác thực người dùng trên giao diện Web quản trị...25
Hình 4.1. Ảnh Project trên Wokwi 26
Hình 4.2. Giao diện khi đăng nhập....33
Hình 4.3. Giao diện khi người dùng nhập sai tài khoản hoặc mật khẩu 33
Hình 4.4. Giao diện Dashboard 34
Hình 4.5. Hộp thoại hiện lên khi giảng viên muốn điểm danh bù cho sinh viên 35
Hình 4.6. Hộp thoại hiện lên khi đăng nhập thành công 36
Hình 4.7. Giao diện Dark Mode 36
Hình 5.1. Hình ảnh sản phẩm khi sẵn sàng quét 38
Hình 5.2. Màn hình sản phẩm hiển thị thông báo khi quét thành công....40
Hình 5.3. Màn hình sản phẩm hiển thị thông báo khi quét thất bại 41

DANH SÁCH BẢNG

Bảng 2.1. Bảng đặc tả kỹ thuật chi tiết của vi điều khiển ESP32 [5].....4
Bảng 2.2. Bảng phân loại và phạm vi tín hiệu của các tần số vô tuyến [10]. 7
Bảng 3.1. Bảng admin_users 13
Bảng 3.2. Bảng sinh_vien.....14
Bảng 3.3. Bảng thiet_bi 15
Bảng 3.4. Bảng lop_hoc 15
Bảng 3.5. Bảng danh_sach_lop 16
Bảng 3.6. Bảng nhat_ky_diem_danh.....17
Bảng 3.7. Danh sách các API Endpoints trong hệ thống.....19
Bảng 5.1. Số liệu tốc độ phản hồi phần cứng ở môi trường mô phỏng.....42
Bảng 5.2. Số liệu đánh giá độ ổn định trong môi trường mô phỏng 43
Bảng 5.3. Số liệu khi kiểm tra hệ thống ở các điều kiện khác nhau trong môi trường mô phỏng 44
Bảng 5.4. Số liệu tốc độ phản hồi phần cứng ở môi trường thực tế.....45
Bảng 5.5. Số liệu đánh giá độ ổn định trong môi trường thực tế 46
Bảng 5.6. Số liệu kiểm tra hệ thống ở các điều kiện khác nhau trong môi trường thực tế 46

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Trong kỷ nguyên số hóa và cuộc cách mạng công nghiệp 4.0, việc chuyển đổi mô hình hướng tới "Đại học thông minh" đang trở thành mục tiêu trọng tâm của các cơ sở giáo dục nhằm nâng cao hiệu quả quản trị. Trong đó, công tác quản lý chuyên cần đóng vai trò then chốt, không chỉ là căn cứ đánh giá kỷ luật và điều kiện dự thi của sinh viên mà còn là thước đo phản ánh chất lượng giảng dạy. Tuy nhiên, phương pháp điểm danh thủ công truyền thống hiện nay đang bộc lộ những bất cập nghiêm trọng: việc gọi tên tuần tự gây lãng phí từ 10-15 phút mỗi tiết học; tình trạng "điểm danh hộ" làm suy giảm kỷ cương học đường; đồng thời dữ liệu ghi chép trên giấy thường rời rạc, dễ phát sinh sai sót và có độ trễ cao trong quá trình nhập liệu, khiến nhà trường không thể nắm bắt tình hình sĩ số theo thời gian thực [1]. Mặt khác, dù một số giải pháp IoT đã bước đầu được đề xuất, nhiều hệ thống vẫn gặp thách thức về độ trễ (latency) và rủi ro xung đột dữ liệu (Race Condition) khi quá phụ thuộc vào hạ tầng kết nối đám mây. Xuất phát từ những bài toán thực tiễn đó, đề tài "Thiết kế hệ thống điểm danh sinh viên tự động sử dụng công nghệ RFID" được thực hiện nhằm xây dựng một giải pháp phân tán ứng dụng xử lý tại vùng biên (Edge Computing). Kiến trúc này không chỉ đảm bảo tốc độ phản hồi tức thì, loại bỏ tình trạng ùn tắc tại cửa lớp mà còn đảm bảo tính minh bạch, đồng bộ và bảo toàn dữ liệu tuyệt đối, góp phần hiện đại hóa toàn diện công tác quản lý giáo dục hiện nay.

1.2. Tình hình nghiên cứu

1.2.1. Các hệ thống trong nước

Tại thị trường trong nước, các giải pháp điểm danh hiện nay chủ yếu tập trung vào ba hướng tiếp cận chính nhưng đều bộc lộ những hạn chế về mặt vận hành. Hệ thống nhận diện vân tay dù có độ chính xác cao nhưng thường gây ùn tắc tại cửa lớp do tốc độ xử lý chậm (2-3 giây/lượt) và thiếu tính thời gian thực do cơ chế trút dữ liệu thủ công qua USB. Các ứng dụng di động kết hợp mã QR động giúp tiết kiệm chi phí phần cứng nhưng dễ bị lợi dụng để điểm danh hộ thông qua việc chia sẻ ảnh chụp mã hoặc sử dụng phần mềm giả

mạo vị trí GPS. Bên cạnh đó, các nghiên cứu về RFID trước đây đa phần chỉ dừng lại ở mức mô hình đơn giản sử dụng Arduino hoặc ESP8266 với dữ liệu lưu trữ cục bộ, chưa xây dựng được hệ thống quản trị tập trung trên nền tảng Web và cơ sở dữ liệu quan hệ hoàn chỉnh để đáp ứng nhu cầu quản lý quy mô lớn.

1.2.2. Các hệ thống nước ngoài

Trên thế giới, các giải pháp điểm danh hiện đại đang được áp dụng rộng rãi với ba hướng tiếp cận chủ đạo: hệ thống thẻ thông minh đa năng (Mifare/NFC) tại Mỹ và Châu Âu đóng vai trò "chìa khóa" toàn diện cho mọi hoạt động trong khuôn viên trường; công nghệ RFID tầm xa cho phép điểm danh hàng loạt khi sinh viên đi qua cửa nhưng có chi phí thiết bị rất cao và dễ bị nhiễu tín hiệu; và hệ thống nhận diện khuôn mặt bằng AI tại Trung Quốc đảm bảo độ chính xác tuyệt đối nhưng đòi hỏi hạ tầng máy chủ đồ họa tốn kém cùng các lo ngại về quyền riêng tư. Tổng hợp các ưu nhược điểm, việc xây dựng hệ thống đọc thẻ RFID băng tần 13.56 MHz kết hợp với các kỹ thuật tối ưu hóa phần mềm như Edge Caching và REST API được xem là lựa chọn tối ưu nhất. Giải pháp này không chỉ đảm bảo hiệu năng xử lý cao, tốc độ phản hồi tức thì mà còn phù hợp với điều kiện kinh tế và hạ tầng kỹ thuật của các trường đại học tại Việt Nam hiện nay.

1.3. Lý do chọn đề tài

Đề tài hướng tới xây dựng một sản phẩm hoàn chỉnh dựa trên 3 lý do cốt lõi sau:

- Ưu thế của công nghệ RFID: Mang lại sự cân bằng tối ưu giữa chi phí thấp, độ bền cao và tốc độ nhận diện tức thời (<0.1s), vượt trội hơn so với QR Code hay sinh trắc học về tính tiện dụng và quyền riêng tư.
- Tự xây dựng Backend**:** Thay vì phụ thuộc vào các dịch vụ bên thứ ba, nhóm tự xây dựng hệ thống máy chủ độc lập với FastAPI (Python) và Supabase, giúp kiểm soát hoàn toàn luồng dữ liệu và khả năng bảo mật.
- Tối ưu hệ thống: Giải quyết triệt để bài toán tốc độ thông qua kỹ thuật Edge Caching (truy xuất tại thiết bị biên) và đảm bảo tính chính xác dữ liệu tuyệt đối bằng cơ chế Mutex Lock (chống lỗi Race Condition).

1.4. Mục tiêu và phạm vi đề tài

1.4.1. Mục tiêu đề tài

- Thiết kế và xây dựng hệ thống điểm danh sinh viên tự động sử dụng công nghệ RFID và vi điều khiển ESP32.
- Xây dựng hệ thống Backend và cơ sở dữ liệu để quản lý thông tin sinh viên, lớp học và dữ liệu điểm danh.
- Phát triển giao diện Web quản trị giúp giảng viên theo dõi, thống kê và quản lý điểm danh.

1.4.2. Phạm vi đề tài

Đề tài tập trung xây dựng hệ thống điểm danh sinh viên sử dụng thẻ RFID tần số 13.56 MHz (chuẩn ISO/IEC 14443A – Mifare Classic 1K) kết hợp với vi điều khiển ESP32 để thu thập và đồng bộ dữ liệu lên máy chủ thông qua mạng Wifi. Hệ thống được triển khai thử nghiệm trên mô hình một phòng học giả định với quy mô từ 50–100 sinh viên nhằm đánh giá thực tế khả năng vận hành và tính khả thi của giải pháp lưu trữ tạm dữ liệu tại thiết bị biên (Edge Caching). Song song đó, đồ án cũng xây dựng hoàn thiện một nền tảng quản trị tập trung, cho phép giảng viên truy xuất các báo cáo chuyên cần trực quan và chính xác từ hệ thống cơ sở dữ liệu đám mây.

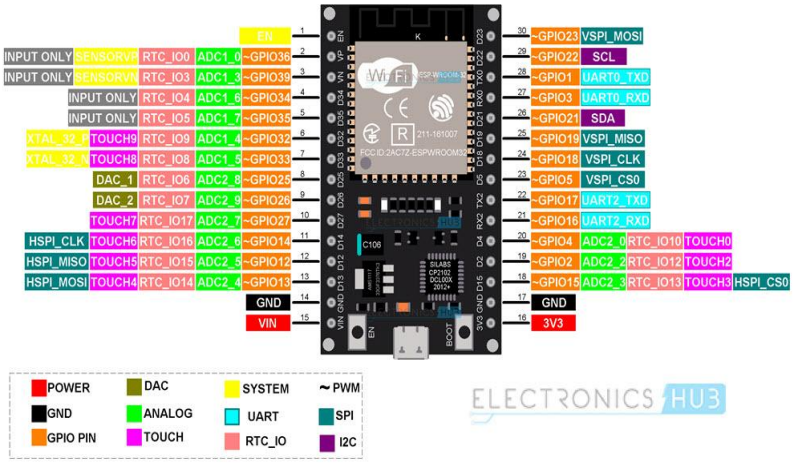
Mặc dù trong phạm vi hiện tại hệ thống vẫn đòi hỏi nguồn điện và kết nối Wifi ổn định để duy trì các chu kỳ đồng bộ, đồng thời chưa mở rộng các tính năng phân tích hành vi nâng cao hay nhận diện khuôn mặt chống gian lận, nhưng kiến trúc phần mềm đã được thiết kế theo hướng module hóa linh hoạt. Điều này đảm bảo hệ thống luôn duy trì sự ổn định của luồng dữ liệu và sẵn sàng cho việc nâng cấp, mở rộng quy mô trong tương lai. Tóm lại, đồ án đã thiết lập thành công quy trình điểm danh tự động khép kín từ thiết bị biên đến quản lý đám mây, chứng minh tính khả thi của giải pháp và tạo tiền đề vững chắc để hướng tới một hệ thống quản lý giáo dục thông minh, toàn diện hơn.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về vi điều khiển ESP32

2.1.1. Kiến trúc và đặc tả kỹ thuật

ESP32 là dòng SoC (System on Chip) mạnh mẽ từ Espressif Systems, sản xuất trên tiến trình 40nm với mức tiêu thụ năng lượng thấp, tối ưu cho các ứng dụng IoT. Trái tim của ESP32 là bộ vi xử lý lõi kép kiến trúc Harvard, mang lại năng lực tính toán vượt trội so với các dòng vi điều khiển 8-bit thông thường. Khả năng này cho phép thiết bị xử lý song song các tác vụ mạng phức tạp, thuật toán mã hóa và phân tích dữ liệu hiệu quả, đáp ứng nhu cầu kết nối liên tục trong môi trường di động và thông minh [2, 3].



Hình 2.1. Sơ đồ chân của bo 30 chân ESP32 DevKit - Ảnh: ELECTRONICS HUB [4].

Dưới đây là bảng thông số kỹ thuật của ESP32:

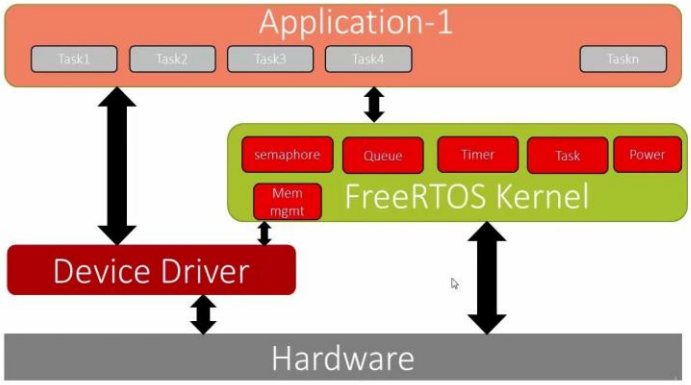
Bảng 2.1. Bảng đặc tả kỹ thuật chi tiết của vi điều khiển ESP32 [5].

Danh mục	Thông số chi tiết
Vi xử lý (CPU)	Dual-core Xtensa® 32-bit LX6; Xung nhịp 80 MHz – 240 MHz (600 DMIPS).
Bộ nhớ nội	448 KB ROM (Boot/Core); 520 KB SRAM (Data/Inst); 16 KB RTC Memory.

Bộ nhớ ngoài	Hỗ trợ SPI Flash lên đến 16 MB.
Kết nối không dây	Wi-Fi 802.11 b/g/n (150 Mbps); Bluetooth Dual-mode (Classic & BLE).
Giao tiếp ngoại vi	34 chân GPIO; 3x UART, 3x SPI, 2x I2C; 12-bit ADC (18 kênh), 8-bit DAC.
Bảo mật	Hỗ trợ chuẩn WPA/WPA2/WPA3 và mã hóa phần cứng.

2.1.2. Hệ điều hành thời gian thực (RTOS)

Trong lĩnh vực phát triển phần mềm nhúng hiện đại, việc tích hợp FreeRTOS trên vi điều khiển ESP32 đóng vai trò then chốt giúp tối ưu hóa hiệu năng và khai thác triệt để sức mạnh của kiến trúc đa lõi. Khác với phương pháp lập trình Bare-metal truyền thống vốn dễ gây nghẽn mạch do thực thi tuần tự, FreeRTOS cho phép chia nhỏ chương trình thành các luồng độc lập (Task) và phân bổ tài nguyên CPU linh hoạt dựa trên mức độ ưu tiên.



Hình 2.2. Hình ảnh minh họa một dự án dùng FreeRTOS

Đặc biệt, nhờ kiến trúc lõi kép, hệ điều hành này hỗ trợ quản lý đa xử lý đối xứng, giúp tách biệt các tác vụ tính toán mạng phức tạp trên Core 0 và các tác vụ điều khiển ngoại vi thời gian thực trên Core 1. Để đảm bảo tính nhất quán và an toàn khi trao đổi thông tin giữa các luồng, cơ chế hàng đợi (Queue) được ứng dụng như một thành phần trung gian

hiệu quả, giúp truyền tải thông điệp nhanh chóng và ngăn chặn hoàn toàn các tranh chấp vùng nhớ phát sinh trong quá trình vận hành [6].

2.2. Công nghệ nhận dạng vô tuyến RFID

2.2.1. Khái niệm và nguyên lý hoạt động

RFID là công nghệ nhận dạng tự động qua sóng vô tuyến, bao gồm Reader và Thẻ.Đối với hệ thống Thẻ thụ động, quy trình vận hành dựa trên Định luật cảm ứng điện từ Faraday: Reader phát trường điện từ.

Hình 2.3. Nguyên lý hoạt động của RFID - Ảnh: Trackify

Khi Thẻ tiến vào vùng phủ sóng, từ thông biến thiên tạo ra suất điện động cảm ứng được chỉnh lưu thành dòng DC để nuôi vi mạch hoạt động. Sau khi được kích hoạt, vi mạch thực hiện kỹ thuật điều chế tải bằng cách thay đổi trở kháng ăng-ten, tạo ra sự biến thiên từ trường tương ứng mà Reader có thể ghi nhận và giải mã thành chuỗi UID để định danh đối tượng chính xác.

2.2.2. Đặc tính tần số và Module MFRC522

Công nghệ RFID hoạt động ở nhiều dải tần, trong đó dải tần HF 13.56 MHz là tiêu chuẩn phổ biến nhất thế giới cho các ứng dụng thẻ thông minh và kiểm soát ra vào. Tần số này sở hữu nhiều ưu điểm vượt trội như tốc độ truyền tải dữ liệu đạt 106 kbps, hỗ trợ các cơ chế mã hóa bảo mật mạnh mẽ và ít bị suy hao tín hiệu trong môi trường có độ ẩm cao.

Để khai thác tối đa lợi thế của dải tần này, hệ thống tích hợp module MFRC522 – một vi mạch chuyên dụng tuân thủ nghiêm ngặt chuẩn quốc tế ISO/IEC 14443A.

Hình 2.4. Module MFRC522

Việc sử dụng module này không chỉ giúp thiết bị đầu cuối trở nên nhỏ gọn mà còn đảm bảo khả năng giao tiếp ổn định với vi điều khiển thông qua giao thức SPI tốc độ cao, giúp triệt tiêu độ trễ khi thực hiện định danh sinh viên. Module MFRC522 là một vi mạch tích hợp cao do NXP Semiconductors sản xuất, chuyên dùng cho giao tiếp không tiếp xúc ở tần số 13.56 MHz. Module này tuân thủ chuẩn quốc tế ISO/IEC 14443 Type A và hỗ trợ giao tiếp với vi điều khiển máy chủ thông qua giao thức SPI với tốc độ truyền tải cực cao [10].

Bảng 2.2. Bảng phân loại và phạm vi tín hiệu của các tần số vô tuyến [10].

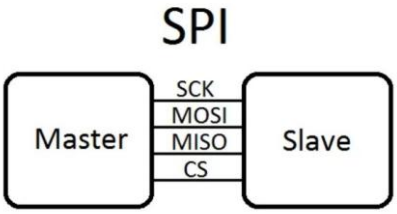
Tần số vô tuyến (Radio Frequency)	Phân loại	Phạm vi truyền tín hiệu
120-125 kHz	Tần số thấp	Ngắn, từ vài cm tới 1 mét
13,56 MHz	Tần số cao	Trung bình, từ vài cm tới vài mét
868 – 928 MHz	Tần số siêu cao	Xa, trên 8 mét
2,45 – 5,8 GHz	Tần số vi sóng	Rất xa, trên 10 mét

2.2.3. Cấu trúc bộ nhớ thẻ Mifare Classic 1K

Dòng thẻ Mifare Classic 1K sở hữu bộ nhớ EEPROM 1024 bytes, được tổ chức chặt chẽ thành 16 Sector, mỗi Sector gồm 4 Block với dung lượng 16 bytes/Block. Trong kiến trúc này, Sector 0 - Block 0 (Manufacturer Block) là phân vùng quan trọng nhất, chứa mã UID 4 bytes duy nhất được thiết lập ở chế độ chỉ đọc để định danh sinh viên trên hệ thống. Bên cạnh đó, tại mỗi Sector, Block 3 (Sector Trailer) đóng vai trò quản lý bảo mật, lưu trữ các khóa Key A, Key B và các bit điều khiển quyền truy cập Đọc/Ghi. Các phân vùng còn lại (Block 0-2 của các Sector từ 1-15) là các Data Blocks dùng để lưu trữ dữ liệu người dùng tùy chọn. Sự phân cấp nghiêm ngặt này không chỉ tối ưu hóa việc quản lý dữ liệu mà còn đảm bảo tính bảo mật cao cho từng phân vùng riêng biệt, đáp ứng tốt yêu cầu của các hệ thống kiểm soát và điểm danh hiện đại [9].

2.2.4. Giao thức ngoại vi SPI và I2C

Tại tầng giao tiếp vật lý, SPI và I2C là hai chuẩn truyền thông nối tiếp chủ đạo giúp kết nối vi điều khiển với ngoại vi. SPI (Full-duplex) sử dụng 4 đường tín hiệu (SCK, MOSI, MISO, SS), sở hữu băng thông lớn nên được ưu tiên cho module RFID để đảm bảo tốc độ định danh tức thời. Trong khi đó, I2C (Half-duplex) hoạt động tối giản với 2 đường tín hiệu (SDA, SCL), cho phép quản lý nhiều thiết bị qua cơ chế địa chỉ, lý tưởng để kết nối các ngoại vi như màn hình LCD nhằm tiết kiệm tối đa chân GPIO. [11, 12]



Hình 2.5. Hình ảnh minh họa giao thức SPI

2.2.5. Giao thức HTTP và Kiến trúc RESTful API

Kiến trúc RESTful API vận hành theo mô hình Khách - Chủ và có tính chất phi trạng thái (Stateless), nghĩa là mỗi yêu cầu từ Client phải chứa đầy đủ thông tin để Server xử lý

độc lập. Hệ thống sử dụng các phương thức chuẩn hóa như GET và POST, đồng thời phản hồi kết quả qua các mã trạng thái chuẩn: 200 OK (thành công), 400 Bad Request (lỗi dữ liệu), 401 Unauthorized (lỗi xác thực) và 500 Internal Server Error (lỗi máy chủ).

2.2.6. Định dạng dữ liệu JSON

Hệ thống tích hợp JSON làm định dạng dữ liệu trung gian trọng lượng nhẹ với cấu trúc Khóa - Giá trị. Quy trình xử lý gồm hai giai đoạn then chốt: Tuần tự hóa (chuyển đổi dữ liệu từ RAM thành chuỗi ký tự để gửi đi) và Giải tuần tự hóa (phân tích chuỗi nhận được để tái tạo cấu trúc dữ liệu ban đầu). Sự phối hợp này đảm bảo tính toàn vẹn và chính xác của thông tin điểm danh khi truyền tải xuyên suốt các lớp kiến trúc [13].

2.3. Công nghệ Web và cơ sở dữ liệu

2.3.1. Hệ quản trị CSDL quan hệ (RDBMS) và tính chất ACID

CSDL đóng vai trò là hạ tầng lưu trữ bền vững, đảm bảo tính toàn vẹn dữ liệu cho hệ thống điểm danh thông qua các RDBMS tiêu biểu như PostgreSQL hay MySQL. Dữ liệu được tổ chức chặt chẽ dưới dạng các bảng, liên kết qua cơ chế Khóa chính (Primary Key) và Khóa ngoại (Foreign Key) nhằm duy trì tính toàn vẹn tham chiếu và ngăn chặn các bản ghi lỗi phát sinh. Để đảm bảo độ tin cậy tuyệt đối, hệ thống tuân thủ nghiêm ngặt mô hình giao dịch ACID (Atomicity, Consistency, Isolation, Durability). Đây là tập hợp các quy tắc giúp bảo vệ dữ liệu trước các sự cố phần cứng hoặc lỗi phần mềm đột xuất, đảm bảo mọi thông tin quẹt thẻ của sinh viên luôn được ghi nhận chính xác, an toàn và hỗ trợ trích xuất báo cáo với hiệu suất cao. [14, 15].

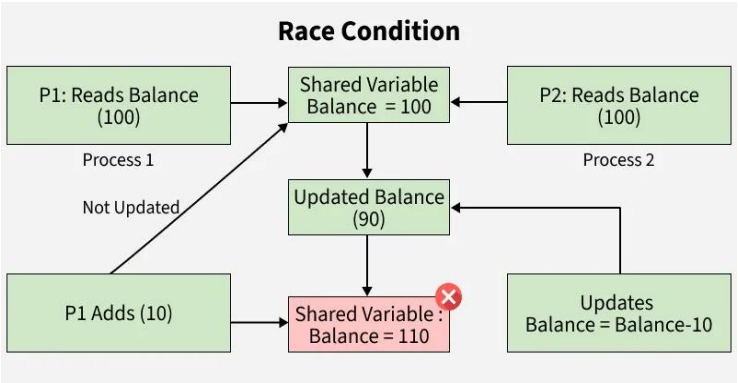
2.3.2. Khung phần mềm Web (Web Framework)

Phần mềm máy chủ đóng vai trò trung gian xử lý logic giữa thiết bị đầu cuối và CSDL. Trong hệ sinh thái ngôn ngữ Python, các Web Framework hiện đại tiêu biểu như FastAPI được thiết kế dựa trên chuẩn ASGI. Điểm khác biệt cốt lõi của các framework này chính là khả năng lập trình bất đồng bộ giúp tối ưu hóa hiệu suất xử lý. Cụ thể, khi hệ thống tiếp nhận các truy vấn tiêu tốn thời gian xử lý như đọc ghi dữ liệu hoặc chờ phản hồi từ CSDL, thay vì giữ CPU ở trạng thái chờ đợi vô ích, framework sẽ giải phóng luồng đó để

phục vụ các yêu cầu mạng khác từ phía người dùng. Kỹ thuật này giúp máy chủ duy trì khả năng xử lý hàng chục ngàn kết nối đồng thời với mức tiêu hao tài nguyên phần cứng cực thấp, đảm bảo tính ổn định và linh hoạt cho hệ thống [16].

2.3.3. Xung đột luồng (Race Condition) và Đồng bộ hóa (Synchronization)

Trong khoa học máy tính, các hệ thống đa luồng thường xuyên phải đối mặt với bài toán tương tranh và hiện tượng xung đột luồng. Hiện tượng Race Condition xảy ra khi có hai hoặc nhiều luồng xử lý cùng chia sẻ và tiến hành sửa đổi một tài nguyên chung, tiêu biểu là các trường dữ liệu trong CSDL, tại cùng một thời điểm cực ngắn. Do CPU thực hiện các lệnh đọc và ghi xen kẽ nhau một cách ngẫu nhiên, dữ liệu cuối cùng được lưu lại sẽ phụ thuộc hoàn toàn vào thứ tự luồng nào hoàn thành sau cùng, dẫn đến tình trạng sai lệch dữ liệu hệ thống.



Hình 2.6. Minh họa Race Condition - Ảnh GeeksforGeeks

Để ngăn chặn triệt để rủi ro này, các hệ điều hành và ngôn ngữ lập trình cung cấp cơ chế đồng bộ hóa bằng khóa loại trừ tương hỗ, hay còn gọi là Mutex Lock. Trong cơ chế này, đoạn mã lệnh thực hiện việc đọc hoặc ghi tài nguyên chung được định nghĩa là khu vực găng. Thuật toán Mutex quy định rằng tại một thời điểm nhất định, hệ thống chỉ cho phép duy nhất một luồng giữ khóa để tiến vào khu vực găng xử lý. Bất kỳ luồng nào đến sau đều bắt buộc phải chuyển sang trạng thái chờ và chỉ khi luồng hiện hành hoàn tất tiến trình rồi thực hiện nhả khóa, luồng tiếp theo mới được phép tiến vào.

CHƯƠNG 3 THIẾT KẾ HỆ THỐNG

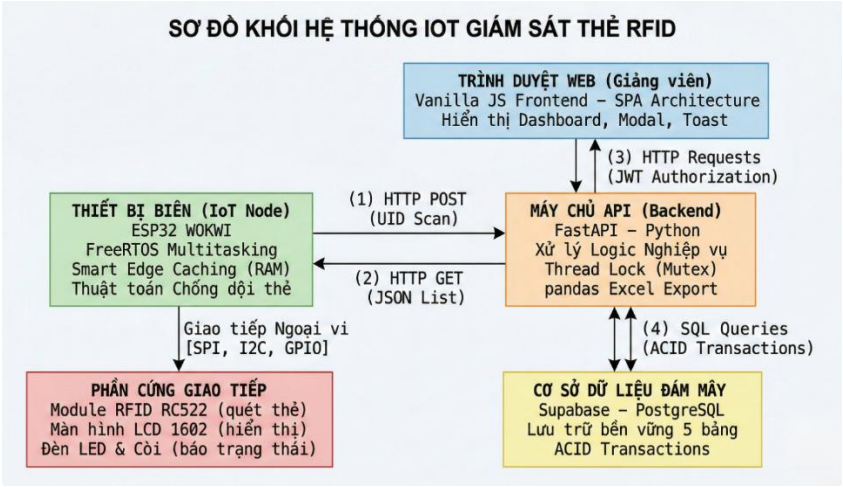
3.1. Phân tích yêu cầu và Sơ đồ khối kiến trúc hệ thống

3.1.1. Phân tích yêu cầu hệ thống

Hệ thống được thiết kế nhằm tối ưu hóa nghiệp vụ điểm danh thông qua việc nhận diện thẻ RFID và phản hồi tức thời trạng thái qua LED, còi báo cùng màn hình LCD. Thiết bị biên có khả năng tự động xác định ca học theo thời gian thực, đồng thời kết nối với giao diện Web để giảng viên quản lý và trích xuất báo cáo chuyên nghiệp. Về hiệu năng, hệ thống đặt mục tiêu độ trễ phản hồi tại biên dưới 0.5 giây nhằm triệt tiêu tình trạng ùn tắc, kết hợp với khả năng xử lý tương tranh tại máy chủ để ngăn chặn tuyệt đối lỗi xung đột dữ liệu. Đặc biệt, cơ chế Edge Caching (lưu trữ đệm tại chỗ) được tích hợp để duy trì hoạt động ổn định ngay cả khi kết nối Wifi chập chờn, đảm bảo tính sẵn sàng cao. Kiến trúc tổng thể hướng đến khả năng mở rộng linh hoạt và tối ưu hóa tài nguyên SRAM trên vi điều khiển, đồng thời ưu tiên tính bảo mật và bảo toàn dữ liệu trong suốt quá trình truyền tải từ thiết bị biên lên đám mây. Sự phối hợp này không chỉ nâng cao trải nghiệm người dùng mà còn đảm bảo tính minh bạch và chính xác tuyệt đối cho dữ liệu giáo vụ.

3.1.2. Sơ đồ khối tổng quát

Hệ thống được xây dựng theo kiến trúc Edge-to-Cloud gồm bốn phân hệ chính gắn kết chặt chẽ, đảm bảo luồng dữ liệu thông suốt từ thực địa lên đám mây. Khối Thiết bị biên (ESP32) đóng vai trò Edge Node thông minh, chịu trách nhiệm thu thập dữ liệu RFID và ứng dụng cơ chế Edge Caching để phản hồi trạng thái tức thì qua màn hình LCD. Dữ liệu từ biên được truyền tới Khối Máy chủ ứng dụng (FastAPI) – "bộ não" điều phối trung tâm – để xử lý logic ca học và kiểm soát đồng bộ luồng dữ liệu. Toàn bộ thông tin sau đó được lưu trữ bền vững tại Khối Cơ sở dữ liệu (PostgreSQL trên nền tảng Supabase), đảm bảo tính bảo mật và toàn vẹn tuyệt đối theo chuẩn ACID. Cuối cùng, Khối Giao diện quản trị cung cấp nền tảng Web trực quan giúp giảng viên giám sát lớp học theo thời gian thực và trích xuất báo cáo tự động. Cấu trúc module hóa này không chỉ tối ưu hóa hiệu suất xử lý mà còn tạo điều kiện thuận lợi cho việc mở rộng quy mô hệ thống trong tương lai.



Hình 3.1. Sơ đồ khối tổng quát của hệ thống

3.1.3. Kiến trúc Bộ đệm thông minh

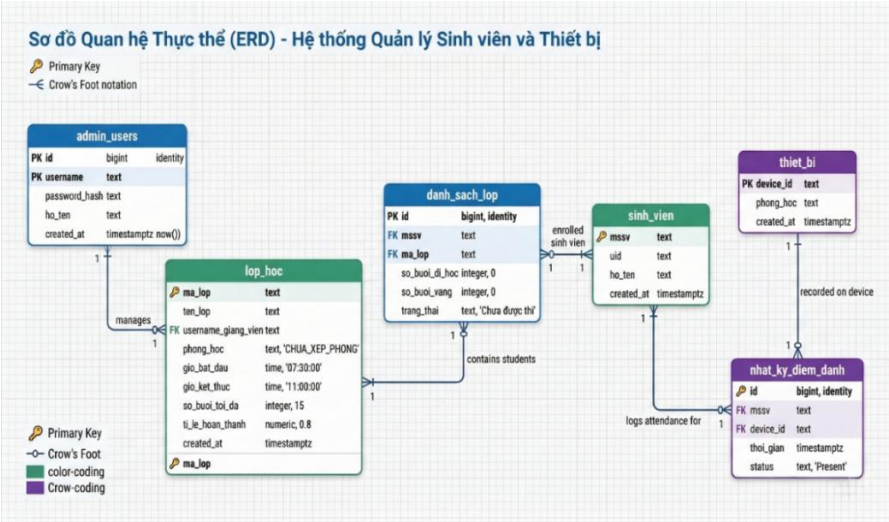
Để tối ưu hóa thời gian phản hồi dưới 0.5 giây và giải quyết giới hạn 520 KB SRAM của ESP32, hệ thống triển khai cơ chế Smart Edge Caching. Thay vì lưu trữ tĩnh, thiết bị thực hiện đồng bộ động qua HTTP GET mỗi 30 phút hoặc khi khởi động; máy chủ sẽ căn cứ vào thời gian thực (UTC+7) và ID thiết bị để trả về chuỗi JSON chứa danh sách sinh viên của ca học hiện hành. Dữ liệu này được giải tuần tự hóa và lưu trực tiếp vào mảng RAM nội bộ (giới hạn 100 sinh viên), cho phép thuật toán tìm kiếm thực thi trong vài mili giây khi quẹt thẻ. Kiến trúc này giúp phản hồi tức thì, triệt tiêu sự phụ thuộc vào độ trễ Internet và đảm bảo hệ thống vận hành ổn định ngay cả ở chế độ ngoại tuyến (Offline mode). Giải pháp không chỉ tiết kiệm băng thông mà còn giúp hệ thống linh hoạt thích ứng với các thay đổi lịch học mà không cần can thiệp vào mã nguồn thiết bị.

3.2. Thiết kế Cơ sở dữ liệu

Hệ thống sử dụng RDBMS PostgreSQL triển khai trên nền tảng Supabase, đảm bảo tính ổn định và khả năng truy cập linh hoạt qua điện toán đám mây. Cấu trúc dữ liệu được thiết kế tuân thủ nghiêm ngặt chuẩn hóa 3NF nhằm loại bỏ dư thừa, tối ưu hiệu suất truy vấn và duy trì tính toàn vẹn tham chiếu giữa các thực thể sinh viên, lớp học và thời khóa biểu. Kiến trúc này không chỉ hỗ trợ mở rộng quy mô linh hoạt mà còn đảm bảo các đặc

tính ACID trong mọi giao dịch ghi nhận nhật ký quẹt thẻ, giúp dữ liệu luôn nhất quán, an toàn và sẵn sàng cho các tác vụ quản lý chuyên cần theo thời gian thực.

3.2.1. Sơ đồ quan hệ thực thể (ERD)



Hình 3.2. Sơ đồ quan hệ thực thể (ERD)

Mô hình dữ liệu bao gồm các thực thể chính: Sinh viên, Lớp học, Thiết bị, Nhật ký điểm danh, Admin users và danh sách lớp.

3.2.2. Đặc tả chi tiết các bảng dữ liệu

Bảng 1: admin_users (Quản lý tài khoản Giảng viên / Quản trị viên)

Bảng này lưu trữ thông tin xác thực của các giảng viên để đăng nhập vào hệ thống Web Dashboard.

Bảng 3.1. Bảng admin_users

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả chức năng
admin_users	TEXT	PRIMARY KEY	Tên đăng nhập của giảng viên
id	BIGINT	GENERATED ALWAYS AS IDENTITY	Khóa phụ tự động tăng định danh duy nhất

password_hash	TEXT	NOT NULL	Chuỗi mật khẩu đã được băm (hashing) bằng thuật toán bcrypt để bảo mật.
ho_ten	TEXT		Họ và tên đầy đủ của giảng viên.
created_at	TIMESTAMPTZ	DEFAULT now()	Mốc thời gian hệ thống tạo tài khoản này.

Bảng 2: sinh_vien (Quản lý thông tin định danh sinh viên): Bảng chứa dữ liệu tĩnh của người học, được sử dụng để đối chiếu với thẻ RFID.

Bảng 3.2. Bảng sinh_vien

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả chức năng
mssv	TEXT		Mã số sinh viên
uid	TEXT		Mã UID vật lý của thẻ RFID gắn với sinh viên này
ho_ten	TEXT	NOT NULL	Họ và tên đầy đủ của sinh viên.
created_at	TIMESTAMPTZ	DEFAULT now()	Mốc thời gian nạp dữ liệu sinh viên vào hệ thống.

Bảng 3: thiet_bi (Quản lý thiết bị IoT đầu cuối): Bảng này đóng vai trò ánh xạ địa chỉ logic của mạch ESP32 với không gian vật lý.

Bảng 3.3. Bảng thiet_bi

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả chức năng
device_id	TEXT	PRIMARY KEY	Mã định danh phần cứng
phong_hoc	TEXT	NOT NULL	Tên phòng học vật lý được lắp đặt thiết bị này
created_at	TIMESTAMPTZ	DEFAULT now()	Thời gian khai báo thiết bị lên hệ thống.

Bảng 4: lop_hoc (Cấu hình Thời khóa biểu và Học phần): Đây là bảng cốt lõi chứa cấu hình thời gian và không gian, giúp Backend tự động nội suy ca học khi thiết bị gửi dữ liệu lên.

Bảng 3.4. Bảng lop_hoc

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả chức năng
ma_lop	TEXT	PRIMARY KEY	Mã lớp học phần duy nhất
ten_lop	TEXT	NOT NULL	Tên môn học hiển thị trên giao diện Web.
username_giang_vien	TEXT	FOREIGN KEY	Liên kết với bảng admin_users, xác định ai là người quản lý lớp này.
phong_hoc	TEXT	NOT NULL, DEFAULT 'CHUA_XEP_PHONG'	Tên phòng học diễn ra môn học. Sẽ được

			Backend đối chiếu với bảng thiet_bi.
gio_bat_dau	TIME	NOT NULL, DEFAULT '07:30:00'	Thời điểm bắt đầu ca học.
gio_ket_thuc	TIME	NOT NULL, DEFAULT '11:00:00'	Thời điểm kết thúc ca học.
so_buoi_toi_da	INTEGER	NOT NULL, DEFAULT 15	Tổng số buổi học theo đề cương môn học.
ti_le_hoan_thanh	NUMERIC | NOT NULL, DEFAULT 0.8		Tỷ lệ đi học bắt buộc để đủ điều kiện dự thi
created_at	TIMESTAMPTZ	DEFAULT now()	Thời gian khai báo thiết bị lên hệ thống.

Bảng 5: danh_sach_lop (Quản lý tiến độ điểm danh)

Bảng trung gian giải quyết mối quan hệ Nhiều - Nhiều giữa Sinh viên và Lớp học.

Bảng 3.5. Bảng danh_sach_lop

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả chức năng
id	BIGINT	PRIMARY KEY, IDENTITY	Khóa định danh tự động tăng.
mssv	TEXT	FOREIGN KEY	Mã số sinh viên (liên kết bảng sinh_vien).

ma_lop	TEXT	FOREIGN KEY	Mã lớp học phần (liên kết bảng lop_hoc).
so_buoi_di_hoc	INTEGER	DEFAULT 0	Bộ đếm tổng số lần điểm danh thành công của sinh viên trong lớp này.
so_buoi_vang	INTEGER	DEFAULT 0	Bộ đếm số buổi vắng mặt.
trang_thai	TEXT	DEFAULT 'Chưa được thi'	Trạng thái cảnh báo học vụ. Hệ thống sẽ tự động cập nhật dựa trên đối chiếu giữa so_buoi_di_hoc và ti_le_hoan_thanh từ bảng lop_hoc. |

Bảng 6: nhat_ky_diem_danh

Lưu trữ toàn bộ lịch sử tương tác của thẻ RFID với thiết bị IoT để phục vụ kiểm toán, dò lỗi và xuất báo cáo thô.

Bảng 3.6. Bảng nhat_ky_diem_danh

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả chức năng
id	BIGINT	PRIMARY KEY, IDENTITY	Khóa tự động tăng mỗi khi có giao dịch.
mssv	TEXT	FOREIGN KEY	Mã sinh viên thực hiện quẹt thẻ (Sẽ bỏ

			trống nếu là thẻ lạ chưa đăng ký).
device_id	TEXT	FOREIGN KEY	Mã thiết bị nhận tín hiệu (liên kết bảng thiet_bi).
thoi_gian	TIMESTAMPTZ	DEFAULT now()	Mốc thời gian thực chính xác khi gói tin điểm danh được hệ thống ghi nhận.
status	TEXT	DEFAULT 'Present'	Trạng thái quẹt thẻ (Có thể là Present khi hợp lệ, Guest khi là thẻ lạ, hoặc Manual Update khi giảng viên tự cộng điểm trên Web).

3.3. Thiết kế Giải thuật và Logic điều khiển

3.3.1. Giải thuật xử lý tại thiết bị biên (Firmware ESP32)

Chương trình điều khiển ESP32 được tối ưu hóa dựa trên hệ điều hành FreeRTOS, sử dụng cơ chế hàng đợi để điều phối dữ liệu nhịp nhàng giữa hai lõi xử lý độc lập. Core 1 đảm nhận luồng nghiệp vụ thực địa, bao gồm việc định kỳ gọi API để đồng bộ danh sách sinh viên vào RAM và quét tín hiệu RFID liên tục với thuật toán chống dội hiệu quả. Khi phát hiện thẻ, thiết bị thực hiện đối soát UID trực tiếp trên bộ nhớ nội bộ để phản hồi trạng thái tức thì qua hệ thống LED, còi báo và màn hình LCD, sau đó đẩy bản ghi vào hàng đợi. Song song đó, Core 0 vận hành tác vụ mạng dưới dạng chạy nền; tác vụ này chỉ kích hoạt khi hàng đợi có dữ liệu để thực hiện đóng gói JSON và gửi lệnh POST về Backend qua giao thức TCP. Sự tách biệt hoàn toàn giữa xử lý logic tại chỗ và truyền tải mạng chính là

chìa khóa giúp hệ thống duy trì độ nhạy cực cao, đảm bảo khả năng điểm danh liên tục và ổn định ngay cả trong môi trường học đường có mật độ lưu thông lớn.

3.3.2. Giải thuật đồng bộ và xử lý logic tại máy chủ Backend

API POST /api/attendance trên FastAPI đóng vai trò trung tâm điều phối logic nghiệp vụ, sử dụng kỹ thuật Mutex (threading.Lock) để xử lý tương tranh và ngăn chặn triệt để lỗi Race Condition. Khi tiếp nhận dữ liệu, máy chủ thực hiện khóa vùng găng (Critical Section) để xác định phòng học qua mã thiết bị, đồng thời nội suy ca học theo thời gian thực (UTC+7). Dựa trên thông tin này, hệ thống đối chiếu UID để ghi nhận trạng thái hiện diện và cập nhật số buổi tích lũy cho sinh viên hợp lệ; các trường hợp khác được lưu trữ dưới dạng dự thính để phục vụ đối soát. Quy trình kết thúc bằng việc chèn bản ghi vào nhật ký điểm danh trước khi giải phóng khóa, đảm bảo tính nhất quán và chính xác tuyệt đối cho mọi báo cáo chuyên cần.

3.4. Thiết kế Giao tiếp API

Hệ thống định nghĩa các API theo chuẩn RESTful, đảm bảo tính bảo mật và dễ dàng mở rộng. Dữ liệu được trao đổi hoàn toàn dưới định dạng JSON.

Bảng 3.7. Danh sách các API Endpoints trong hệ thống

Endpoint / Đường dẫn	Phương thức	Dữ liệu đầu vào (Payload/Query)	Chức năng chi tiết
/api/students/{device_id}	GET	Tham số đường dẫn: device_id	Xác định ca học và trả về danh sách sinh viên (JSON) để ESP32 lưu đệm.
/api/attendance	POST	JSON Body: {uid, device_id, time_scan}	Ghi nhận quẹt thẻ, xử lý logic điểm danh và đồng bộ luồng dữ liệu.

/token	POST	Form Data: username, password	Xác thực giảng viên (bcrypt) và cấp phát mã JWT (hiệu lực 30 phút)
/my-classes	GET	Header Authorization: Bearer <token>	Giải mã JWT và trả về danh sách các lớp học do giảng viên phụ trách.
/stats	GET	Query Parameter: ?ma_lop=...	Truy xuất bảng View thống kê để lấy toàn bộ dữ liệu hiện diện/vắng mặt của một lớp học cụ thể, phục vụ hiển thị lên Web.
/update-attendance	POST	JSON Body: {mssv, ma_lop, action}	Cấp quyền cho giảng viên can thiệp thủ công (Cộng/Trừ buổi học cho sinh viên quên thẻ). Ghi lại log với nhãn Manual Update.
/export	GET	Query Parameter: ?ma_lop=...	Hệ thống sử dụng thư viện pandas để chuyển đổi dữ liệu CSDL thành định dạng Excel nhị phân (.xlsx) và trả về cho trình duyệt tải xuống.

3.5. Thiết kế Giao diện Web Quản trị (Frontend)

3.5.1. Kiến trúc trang đơn

Nhằm mang lại trải nghiệm phần mềm mượt mà, ứng dụng được xây dựng theo cấu trúc SPA một phần sử dụng JavaScript thuần (Vanilla JS) và Fetch API. Thay vì tải lại (reload) toàn bộ trang web mỗi khi giảng viên chuyển đổi lớp học hoặc bấm nút "Làm mới", hệ thống chỉ gửi yêu cầu nền lên API /stats và tiến hành kết xuất lại (re-render) dữ liệu bên trong vùng nội dung bảng (Table DOM).

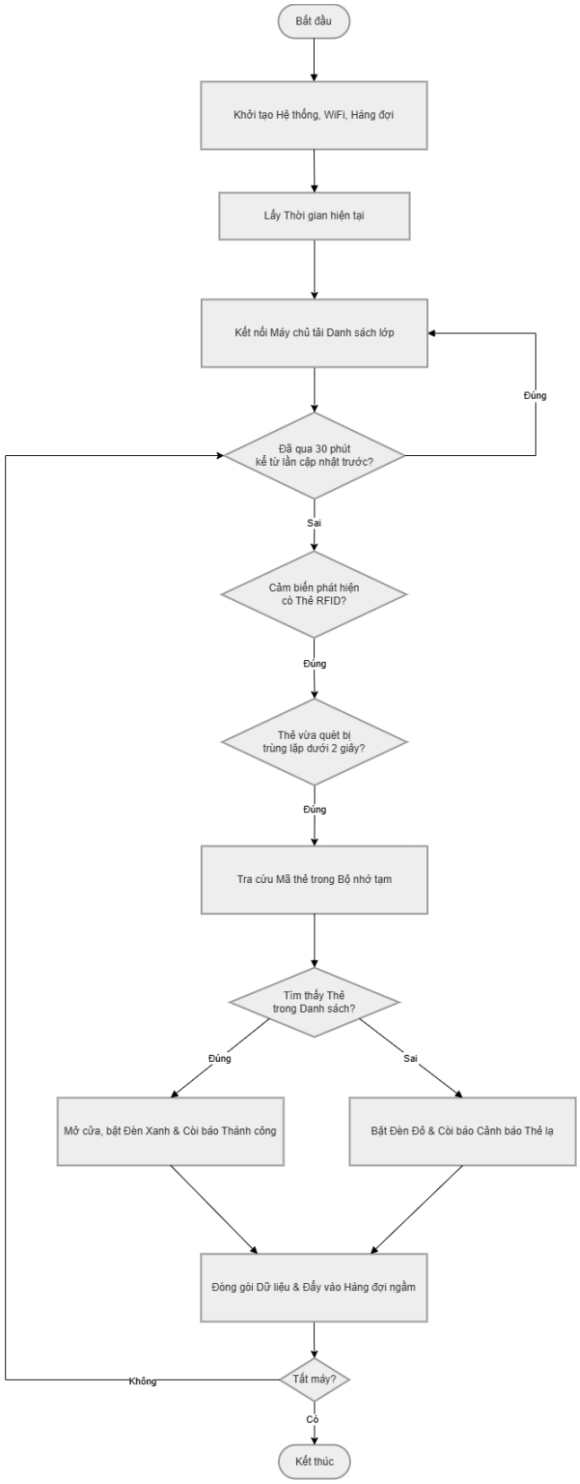
3.5.2. Thiết kế luồng tương tác người dùng (UI/UX)

Giao diện hệ thống được phân lớp chức năng rõ ràng, chú trọng vào tính trực quan và tương tác tối ưu. Lớp xác thực sử dụng mã JWT để bảo mật phiên làm việc, tích hợp các hiệu ứng cảnh báo sinh động nhằm tăng cường trải nghiệm người dùng. Bảng điều khiển trung tâm cung cấp số liệu thống kê sĩ số trực quan, hỗ trợ giảng viên linh hoạt thay đổi giao diện Sáng/Tối, điều chỉnh điểm danh trực tiếp và trích xuất báo cáo Excel chỉ với một thao tác duy nhất. Đặc biệt, thiết kế Responsive giúp hệ thống tương thích hoàn hảo trên cả máy tính lẫn điện thoại, kết hợp cùng các thành phần tùy chỉnh như Modal xác nhận và Toast thông báo giúp luồng tương tác trở nên mượt mà, chuyên nghiệp và thẩm mỹ hơn.

3.6. Lưu đồ thuật toán

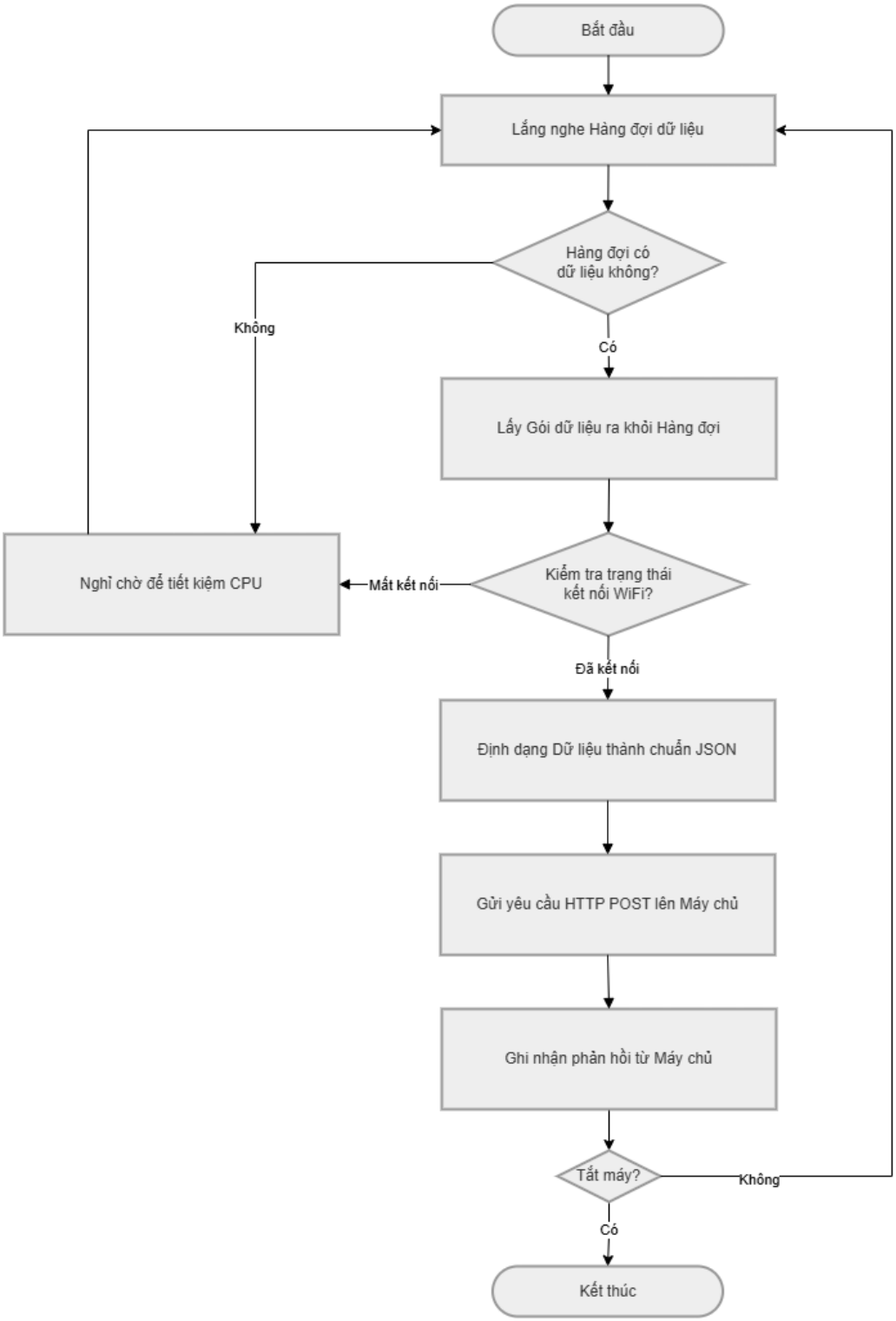
Hệ thống được mô hình hóa qua bốn lưu đồ trọng tâm, thể hiện sự phối hợp chặt chẽ giữa biên và đám mây. Trước hết, lưu đồ xử lý chính tại ESP32 (Core 1) tập trung vào tương tác thực địa và Edge Caching, cho phép đối soát UID trực tiếp trên RAM để phản hồi tức thì. Song song đó, lưu đồ truyền tải mạng ngầm (Core 0) vận hành dựa trên cơ chế bất đồng bộ của FreeRTOS, chuyên giám sát hàng đợi (Queue) để gửi dữ liệu JSON mà không gây gián đoạn luồng điểm danh chính. Tại trung tâm, lưu đồ xử lý tại máy chủ sử dụng khóa Mutex để bảo vệ vùng găng, loại bỏ rủi ro Race Condition. Cuối cùng, lưu đồ xác thực đăng nhập cụ thể hóa quy trình bảo mật qua mã JWT và phản hồi UI/UX trên Dashboard, mang lại sự tin cậy cho công tác quản trị.

3.6.1. Lưu đồ xử lý chính của ESP32 (Core 1 - Quét thẻ & Hiển thị)



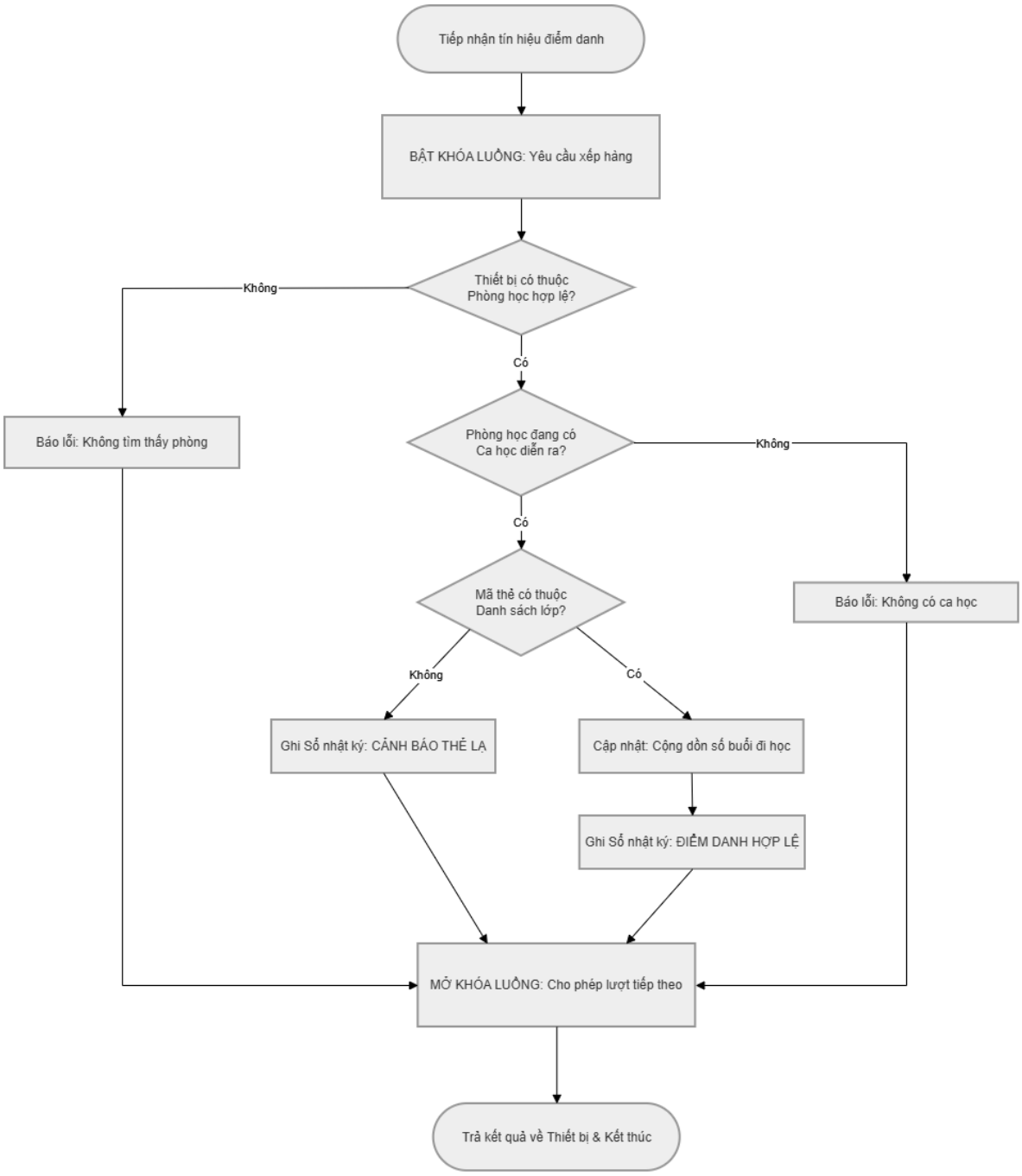
Hình 3.3. Lưu đồ thuật toán xử lý nhận dạng và hiển thị trên Core 1 của vi điều khiển ESP32

3.6.2. Lưu đồ truyền tải mạng ngầm ESP32 (Core 0 - TaskUpload)



Hình 3.4. Lưu đồ thuật toán tác vụ truyền tải mạng ngầm (TaskUpload) trên Core 0

3.6.3. Lưu đồ thuật toán xử lý điểm danh trên Server (FastAPI)



Hình 3.5. Lưu đồ thuật toán xử lý điểm danh và cơ chế đồng bộ luồng (Mutex Lock) trên máy chủ Backend

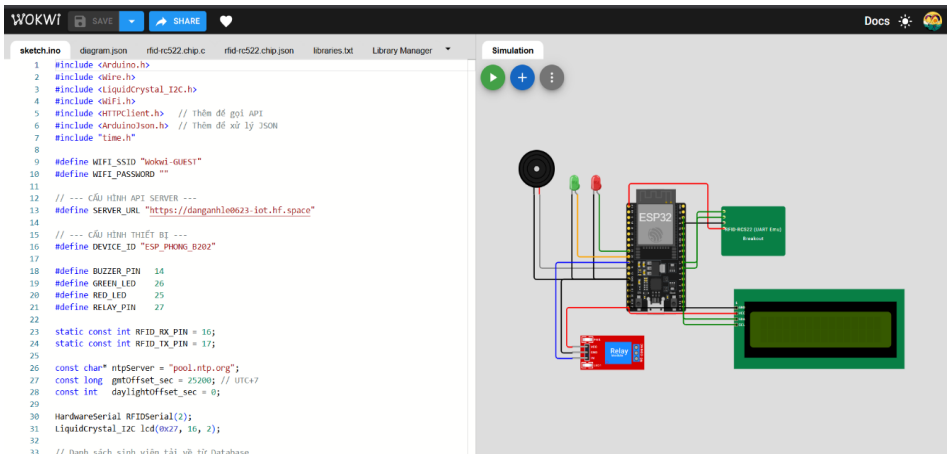
3.6.4. Lưu đồ xác thực đăng nhập Giao diện Web (Frontend)

Hình 3.6. Lưu đồ luồng tương tác và xác thực người dùng trên giao diện Web quản trị.

CHƯƠNG 4 XÂY DỰNG VÀ THIẾT LẬP MÔ PHỎNG

4.1. Môi trường phát triển

Quá trình xây dựng hệ thống được thực hiện đồng bộ trên cả phương diện phần cứng và phần mềm nhằm đảm bảo tính nhất quán từ giai đoạn thử nghiệm đến triển khai thực tế. Về phần cứng, nền tảng Wokwi được sử dụng để mô phỏng và kiểm thử logic của vi điều khiển ESP32 cùng các ngoại vi một cách trực quan trước khi lắp ráp. Về phần mềm, hệ thống Backend được phát triển trên ngôn ngữ Python 3.12, tích hợp cơ sở dữ liệu đám mây Supabase và bảo mật tuyệt đối qua tệp cấu hình .env. Để tối ưu hóa quy trình vận hành, toàn bộ ứng dụng được đóng gói bằng Docker, giúp đồng bộ hóa môi trường giữa máy tính phát triển và máy chủ thực tế. Cuối cùng, hệ thống được triển khai trực tuyến trên nền tảng Hugging Face Spaces, kết hợp với Git để quản lý phiên bản hiệu quả, đảm bảo tính sẵn sàng cao và khả năng vận hành liên tục trong môi trường Internet.



Hình 4.1. Ảnh Project trên Wokwi

4.2. Xây dựng Firmware (Wokwi) cho ESP32

Phần mềm điều khiển (Firmware) của hệ thống được phát triển trên nền tảng môi trường tích hợp Arduino IDE, sử dụng ngôn ngữ lập trình C++. Điểm đột phá về mặt phần mềm của kiến trúc này nằm ở việc ứng dụng triệt để mô hình lập trình hướng sự kiện kết hợp với khả năng xử lý đa nhiệm, qua đó tối ưu hóa năng lực tính toán của vi điều khiển. Giải pháp kỹ thuật này đảm bảo thiết bị đầu cuối luôn duy trì trạng thái hoạt động ổn định

và có độ tin cậy cao, ngay cả trong các điều kiện môi trường mạng viễn thông có độ trễ lớn hoặc thiếu ổn định.

4.2.1. Cấu trúc điều khiển đa nhiệm (FreeRTOS)

Trong các hệ thống nhúng truyền thống, việc sử dụng luồng xử lý tuần tự thường dẫn đến rủi ro tắc nghẽn hệ thống (blocking) khi thiết bị buộc phải chờ đợi phản hồi từ ngoại vi hoặc mạng Internet. Để khắc phục triệt để hạn chế này, chương trình điều khiển đã được thiết kế dựa trên các cơ sở cốt lõi của hệ điều hành thời gian thực FreeRTOS, cụ thể là việc ứng dụng cấu trúc dữ liệu hàng đợi (Queue) và phân mảnh tác vụ (Task) độc lập. Việc khởi tạo cấu trúc uploadQueue và tiến trình TaskUpload đóng vai trò then chốt trong việc cô lập không gian xử lý giữa luồng nghiệp vụ quét thẻ và luồng truyền tải dữ liệu. Cụ thể, khi có sự kiện thẻ RFID được nhận diện, luồng xử lý chính sẽ ngay lập tức đẩy dữ liệu định danh vào hàng đợi và kích hoạt các ngoại vi phản hồi giao diện như màn hình LCD theo thời gian thực. Cùng lúc đó, tiến trình gửi yêu cầu HTTP POST đến máy chủ API sẽ được thực thi hoàn toàn ngầm bởi Core 0 của vi điều khiển. Sự phân luồng này giúp toàn bộ hệ thống loại bỏ hiện tượng treo tạm thời (đơ máy) trong khoảng thời gian thiết lập kết nối mạng.

4.2.2. Cơ chế Local Caching và đồng bộ hóa API

Nhằm đáp ứng yêu cầu khắt khe về thời gian phản hồi với độ trễ (Latency) cam kết dưới 1 giây, hệ thống ESP32 được thiết kế để không phụ thuộc vào chu trình xác thực trực tuyến từ máy chủ trung tâm. Trái lại, thiết bị triển khai kỹ thuật lưu trữ đệm cục bộ (Local Caching), thực hiện thuật toán đối soát trực tiếp trên tập dữ liệu danh sách sinh viên đã được nạp sẵn vào bộ nhớ SRAM nội bộ. Thông qua hàm fetchStudentList(), thiết bị sẽ chủ động khởi tạo giao thức truyền thông với hệ quản trị cơ sở dữ liệu Supabase thông qua nền tảng RESTful API ngay tại chu kỳ khởi động, và thực hiện tái đồng bộ hóa tự động sau mỗi 30 phút. Cơ chế đồng bộ dữ liệu tĩnh này không chỉ giúp tối ưu hóa băng thông mạng mà còn cung cấp cho thiết bị khả năng tự chủ vận hành. Nhờ đó, quá trình định danh và đối soát thông tin sinh viên vẫn diễn ra chính xác và liên tục ngay cả trong kịch bản kết nối

Internet bị gián đoạn, hình thành nên cơ chế hoạt động ngoại tuyến (Offline) dự phòng hoàn hảo.

4.2.3. Xử lý logic nhận dạng và phản hồi người dùng

Quy trình xử lý chuỗi định danh (UID) thu thập từ module viễn thông RFID được thiết kế với tính chặt chẽ cao, nhằm mục đích loại trừ hoàn toàn các xung đột hoặc hiện tượng ghi nhận dữ liệu trùng lặp trong một chu kỳ thời gian ngắn. Thuật toán kiểm duyệt thông qua biến lưu trữ lastUid đóng vai trò như một bộ lọc tín hiệu, tự động triệt tiêu các yêu cầu quét thẻ lặp lại trong ngưỡng thời gian 2 giây. Ngay khi dữ liệu vượt qua bước xác thực ban đầu, hệ thống tiến hành đối soát tuyến tính với danh sách sinh viên nội bộ. Tùy thuộc vào kết quả định danh, các hàm điều khiển trạng thái accessGranted và accessDenied sẽ được kích hoạt để phối hợp hoạt động của các thiết bị ngoại vi, bao gồm hệ thống cảnh báo âm thanh (Buzzer), chỉ thị quang học (LED) và rơ-le (Relay) nhằm thông báo trạng thái trực quan đến người dùng. Kết thúc quy trình nhận diện, tập dữ liệu thô sẽ được cấu trúc hóa thành định dạng tiêu chuẩn JSON trước khi được phân luồng vào hàng đợi (Queue) để chờ tiến trình mạng xử lý.

4.2.4. Đồng bộ thời gian thực qua NTP

Nhằm đảm bảo tính toàn vẹn và giá trị pháp lý của dữ liệu điểm danh, phục vụ đắc lực cho công tác hậu kiểm và đối soát hệ thống, vi điều khiển ESP32 được thiết lập cơ chế đồng bộ hóa thời gian liên tục với các máy chủ thời gian chuẩn quốc tế thông qua mạng Internet. Việc ứng dụng giao thức mạng NTP (Network Time Protocol) cho phép thiết bị trích xuất chính xác thời gian hệ thống. Cơ chế này đóng vai trò tiên quyết trong việc gắn nhãn thời gian thực (Timestamp) cho từng bản ghi điểm danh được tạo ra. Nhờ sự đồng bộ tuyệt đối về mặt thời gian, hệ thống máy chủ ứng dụng sở hữu đầy đủ cơ sở dữ liệu xác thực để tiến hành đối chiếu, phân tích và ánh xạ chính xác thời điểm quét thẻ với thời khóa biểu đã được số hóa trong hệ quản trị cơ sở dữ liệu PostgreSQL.

4.3. Xây dựng Backend

Phân hệ máy chủ ứng dụng (Backend) được thiết kế và triển khai nhằm đảm nhận vai trò cầu nối dữ liệu trọng yếu giữa các thiết bị đầu cuối (Edge Devices) và nền tảng quản trị người dùng. Việc xây dựng Backend không chỉ dừng lại ở chức năng tiếp nhận tín hiệu vật lý thô từ phần cứng mà còn bao trùm các tiến trình phân tích, chuẩn hóa dữ liệu, thực thi logic nghiệp vụ và quản trị cơ sở dữ liệu tập trung. Mục tiêu cốt lõi của giai đoạn này là thiết lập một hạ tầng phần mềm hoạt động bền bỉ, sở hữu năng lực mở rộng cao và bảo đảm tuyệt đối tính toàn vẹn cũng như sự an toàn cho toàn bộ luồng thông tin điểm danh của hệ thống.

4.3.1. Tổng quan kiến trúc Backend

Trong tổng thể kiến trúc hệ thống Vạn vật kết nối (IoT), máy chủ ứng dụng vận hành như một trung tâm điều phối hạt nhân. Phân hệ này chịu trách nhiệm thực thi các logic nghiệp vụ phức tạp, quản lý kết nối với cơ sở dữ liệu và thiết lập các giao diện lập trình ứng dụng theo tiêu chuẩn RESTful (RESTful API), qua đó phục vụ quá trình giao tiếp đồng thời cho cả thiết bị phần cứng ngoại vi (vi điều khiển ESP32) và nền tảng Web quản trị. Về mặt nền tảng công nghệ, hệ thống được phát triển chủ yếu dựa trên ngôn ngữ lập trình Python kết hợp với khung phần mềm (framework) FastAPI. Khung phần mềm này được ưu tiên lựa chọn nhờ khả năng xử lý luồng dữ liệu bất đồng bộ (Asynchronous) mang lại hiệu suất truy xuất vượt trội, đồng thời tích hợp sẵn tiêu chuẩn OpenAPI phục vụ cho việc tự động hóa tài liệu và sở hữu cơ chế xác thực dữ liệu chặt chẽ thông qua thư viện Pydantic. Đối với hạ tầng lưu trữ, đề tài tận dụng nền tảng điện toán đám mây Supabase để triển khai hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở PostgreSQL, sử dụng thư viện supabase-py để thiết lập các tương tác trực tiếp với cơ sở dữ liệu. Nhằm đáp ứng các tiêu chuẩn khắt khe về an toàn thông tin, kiến trúc bảo mật của hệ thống được củng cố bằng phương thức xác thực JSON Web Token (JWT) tích hợp thuật toán mã hóa HS256 để bảo vệ các điểm cuối (Endpoints) của giao diện Web. Song song đó, hệ thống sử dụng thư viện passlib để thực

hiện băm (hashing) mật khẩu của giảng viên, đảm bảo an toàn tuyệt đối cho các luồng dữ liệu truy cập nhạy cảm.

4.3.2. Thiết kế cơ sở dữ liệu (Database Schema)

Hệ thống cơ sở dữ liệu được xây dựng theo mô hình quan hệ chuẩn hóa cao với sáu bảng thực thể cốt lõi, tập trung vào việc phân tách logic giữa Thiết bị (thiet_bi) và Lớp học (lop_hoc). Sự liên kết giữa hai thực thể này được thiết lập động qua thuộc tính "Phòng học" và "Thời gian thực", cho phép vi mạch ESP32 tự động ánh xạ chính xác với ca học đang diễn ra tại thực địa. Bảng lop_hoc không chỉ quản lý thông tin học thuật mà còn tích hợp các tham số cấu hình như số buổi học tối đa và tỷ lệ chuyên cần bắt buộc, tạo cơ sở để hệ thống tự động đánh giá tư cách dự thi của sinh viên. Dữ liệu định danh sinh viên và mã thẻ RFID (uid) được quản lý tập trung tại bảng sinh_vien, kết nối với các lớp học phần thông qua bảng trung gian danh_sach_lop. Phân hệ này ứng dụng các hàm kích hoạt (Trigger) vận hành ngầm trong PostgreSQL để tự động nội suy trạng thái học vụ (như "Được thi" hoặc "Chưa được thi") dựa trên số buổi hiện diện thực tế. Cuối cùng, bảng nhat_ky_diem_danh đóng vai trò là kho lưu trữ lịch sử tương tác (Audit logs) minh bạch với các nhãn trạng thái cụ thể như *Present*, *Guest* hoặc *Manual Update*, đảm bảo tính đối soát và bảo toàn dữ liệu tuyệt đối cho toàn bộ hệ thống.

4.3.3. Xây dựng các API giao tiếp với ESP32

Hệ thống triển khai hai nhóm API cốt lõi đóng vai trò là "xương sống" thiết lập luồng dữ liệu hai chiều giữa thiết bị biên và đám mây. API GET /api/students/{device_id} thực hiện nhiệm vụ đồng bộ danh sách sinh viên theo chu kỳ 30 phút; tại đây, máy chủ tự động nội suy ca học dựa trên mã định danh thiết bị và thời gian thực (UTC+7) để trả về gói tin JSON tối ưu chứa thông tin (UID, MSSV, Họ tên) của lớp học đang diễn ra. Song song đó, tiến trình ghi nhận chuyên cần được đảm nhiệm bởi API POST /api/attendance với cơ chế kiểm soát luồng khắt khe. Để triệt tiêu hoàn toàn rủi ro xung đột dữ liệu (Race Condition) khi sinh viên quẹt thẻ liên tục với tốc độ cao, máy chủ ứng dụng kỹ thuật Mutex Lock (threading.Lock), buộc các yêu cầu phải xếp hàng xử lý tuần tự khi truy cập vào vùng găng

(Critical Section). Quy trình này không chỉ chuẩn hóa thời gian quẹt thẻ từ định dạng Epoch về múi giờ địa phương mà còn thực hiện đối soát UID thông minh: tự động tăng số buổi tích lũy cho sinh viên hợp lệ (*Present*) hoặc ghi nhận cảnh báo an ninh cho thẻ lạ (*Guest*). Việc luôn phản hồi mã thành công HTTP 200 ngay cả với thẻ lạ giúp thiết bị biên giải phóng hàng đợi kịp thời, đảm bảo hệ thống vận hành thông suốt và dữ liệu luôn được bảo toàn tuyệt đối.

4.3.4. Xây dựng các API phục vụ quản lý (Web Dashboard)

Máy chủ Backend cung cấp nhóm API quản lý chuyên biệt với cơ chế bảo mật nghiêm ngặt nhằm hỗ trợ giám sát trực quan trên nền tảng Web Dashboard. Quy trình xác thực bắt đầu tại điểm cuối POST /token, sử dụng thuật toán bcrypt để kiểm tra mật khẩu và cấp phát mã thông báo JWT, đảm bảo mọi truy vấn nghiệp vụ sau đó đều được kiểm soát chặt chẽ qua lớp bảo mật get_current_user. Thông qua các điểm cuối như GET /stats, giảng viên có thể dễ dàng theo dõi thống kê hiện diện và trạng thái điều kiện dự thi của từng sinh viên theo thời gian thực. Đặc biệt, hệ thống tích hợp API GET /export sử dụng thư viện pandas để tự động chuyển đổi dữ liệu từ PostgreSQL sang định dạng Excel, giúp tối ưu hóa tối đa công tác báo cáo chuyên cần cuối kỳ. Ngoài ra, để xử lý các tình huống ngoại lệ (như quên thẻ), API POST /update-attendance cho phép giảng viên can thiệp điều chỉnh thủ công, đồng thời mọi thao tác đều được lưu vết minh bạch dưới nhãn kiểm toán WEB_MANUAL. Giải pháp này đảm bảo tính đối soát rạch ròi giữa dòng dữ liệu tự động từ thiết bị biên và các thao tác quản trị chủ quan, duy trì độ tin cậy và tính toàn vẹn tuyệt đối cho cơ sở dữ liệu học vụ.

4.4. Xây dựng giao diện Web

Phân hệ giao diện người dùng (Frontend) được thiết kế nhằm đảm nhận vai trò cầu nối tương tác trực tiếp giữa đội ngũ giảng viên, nhà quản trị và cơ sở dữ liệu lõi của hệ thống điểm danh. Quá trình xây dựng giao diện tuân thủ nghiêm ngặt các tiêu chuẩn khắt khe về hiệu năng phần mềm, trong đó đặt trọng tâm vào tốc độ tải trang tối ưu, khả năng

đáp ứng linh hoạt trên đa dạng kích thước thiết bị hiển thị (Responsive Design) và việc kiến tạo một trải nghiệm người dùng (UX/UI) trực quan, mang đậm tính hiện đại.

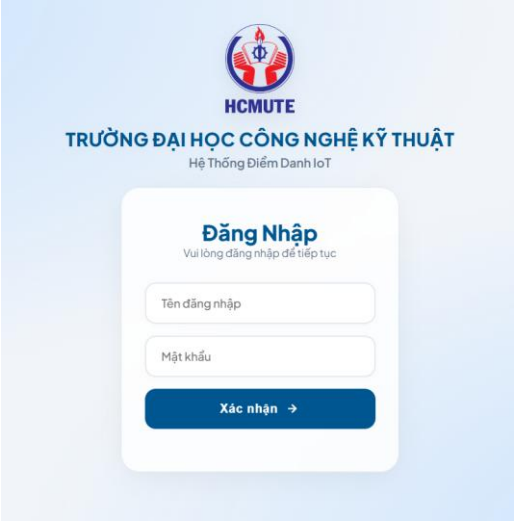
4.4.1. Lựa chọn công nghệ và Kiến trúc Frontend

Nhằm tối ưu hóa tài nguyên và đảm bảo tính gọn nhẹ cho kiến trúc phần mềm, hệ thống quyết định không phụ thuộc vào các thư viện hoặc khung làm việc (framework) cồng kềnh như ReactJS hay VueJS. Thay vào đó, toàn bộ kiến trúc Frontend được phát triển thuần túy dựa trên nền tảng công nghệ web cốt lõi bao gồm HTML5, CSS3 và JavaScript (Vanilla JS). Lựa chọn chiến lược này không chỉ giúp tối thiểu hóa dung lượng của ứng dụng mà còn tạo điều kiện thuận lợi cho việc triển khai linh hoạt trên các máy chủ lưu trữ tĩnh (Static Hosting) mà không đòi hỏi các quy trình cấu hình môi trường phức tạp. Về phương diện thẩm mỹ và thiết kế giao diện (UI), ứng dụng áp dụng ngôn ngữ thiết kế Glass Morphism, đặc trưng bởi các hiệu ứng kính mờ tinh tế, kết hợp đồng bộ với kỹ thuật quản lý biến cục bộ (CSS Variables) được khai báo tập trung tại thẻ :root. Sự tích hợp của hệ thống biến CSS này đóng vai trò then chốt trong việc kiểm soát động các dải màu, đồng thời cung cấp nền tảng kỹ thuật vững chắc để triển khai tính năng chuyển đổi chủ đề hiển thị Sáng/Tối một cách mượt mà.

4.4.2. Xây dựng trang Xác thực người dùng

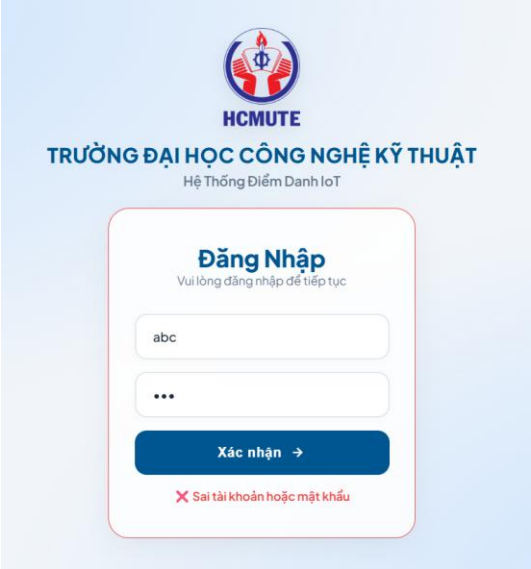
Trong bối cảnh an toàn thông tin được đặt lên hàng đầu, cơ chế bảo mật dữ liệu trở thành điều kiện tiên quyết đối với mọi kiến trúc phần mềm. Do vậy, toàn bộ không gian làm việc của ứng dụng được thiết lập một rào cản xác thực nghiêm ngặt ngay tại trang chủ. Về mặt thiết kế trực quan, biểu mẫu đăng nhập được bố trí tại không gian trung tâm màn hình, áp dụng kỹ thuật đổ bóng mờ (box-shadow) kết hợp hiệu ứng làm nhòe nền (backdrop-filter: blur) nhằm tạo chiều sâu và sự tập trung cho luồng tương tác. Đi sâu vào phương diện tối ưu hóa trải nghiệm người dùng (UX), hệ thống loại bỏ các thông báo lỗi văn bản thông thường, thay vào đó tích hợp hiệu ứng rung lắc vật lý ảo (shakeError) kèm theo sự thay đổi sắc độ viền cảnh báo thành màu đỏ khi phát hiện thông tin xác thực không hợp lệ.

Cơ chế phản hồi thị giác này giúp điều hướng sự chú ý của người dùng một cách trực quan và tinh tế mà không làm gián đoạn mạch làm việc.



Hình 4.2. Giao diện khi đăng nhập

Xét về khía cạnh logic điều khiển (JavaScript), toàn bộ quy trình xác thực được kiểm soát tập trung thông qua hàm handleLogin(). Hàm này chịu trách nhiệm thu thập thông tin định danh, cấu trúc hóa dữ liệu vào đối tượng URLSearchParams và khởi tạo một luồng yêu cầu giao thức HTTP POST hướng tới điểm cuối API /token trên máy chủ.

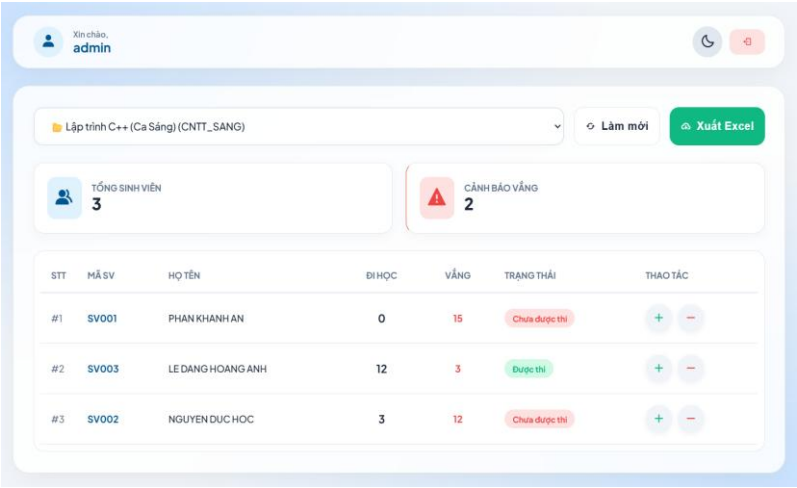


Hình 4.3. Giao diện khi người dùng nhập sai tài khoản hoặc mật khẩu

Khi quá trình đối soát tại Backend hoàn tất thành công, hệ thống sẽ tiếp nhận một mã thông báo bảo mật định dạng JSON Web Token (JWT). Mã thông báo này ngay lập tức được luồng thực thi của trình duyệt mã hóa và lưu trữ an toàn trong không gian bộ nhớ cục bộ (localStorage), thiết lập cơ chế duy trì phiên làm việc liên tục cho giảng viên ở các lần truy cập tiếp theo mà không đòi hỏi quá trình tái đăng nhập thủ công.

4.4.3. Xây dựng Bảng điều khiển (Dashboard) và Quản lý lớp học

Ngay sau khi quy trình xác thực thành công, hệ thống sẽ tự động kích hoạt hiển thị Bảng điều khiển (Dashboard) thông qua việc gỡ bỏ thuộc tính ẩn trên giao diện người dùng. Không gian làm việc chính này được phân bổ khoa học thành ba phân khu chức năng biệt lập nhằm tối ưu hóa luồng công việc của giảng viên. Thanh điều hướng phía trên (Top Navbar) đảm nhiệm vai trò hiển thị định danh người dùng từ localStorage cùng các tiện ích như chuyển đổi chủ đề (Sáng/Tối) và đăng xuất. Tiếp đến, khu vực điều khiển trung tâm tích hợp danh sách thả xuống được nạp dữ liệu động từ API /my-classes kết hợp với các công cụ thao tác nhanh như làm mới dữ liệu và trích xuất báo cáo Excel.



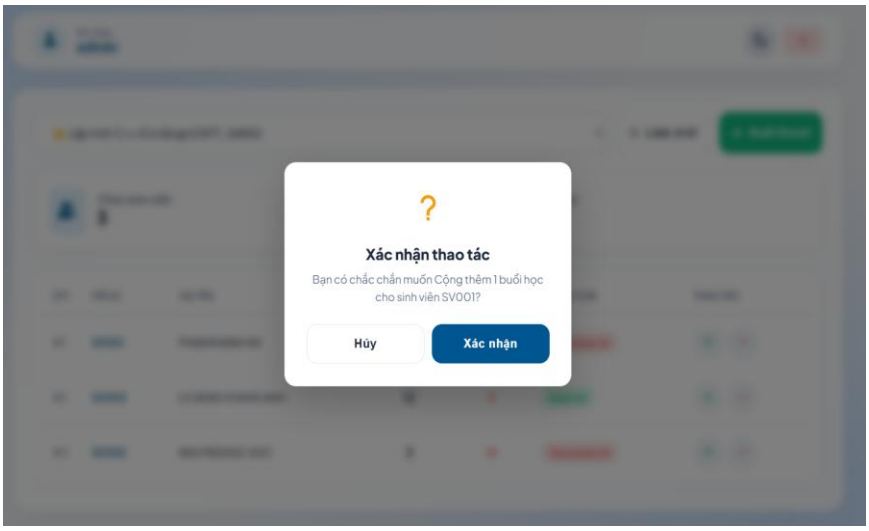
Hình 4.4. Giao diện Dashboard

Cuối cùng, phân khu hiển thị và thống kê đóng vai trò trình diễn dữ liệu trực quan, cung cấp cái nhìn tổng thể về sĩ số và các cảnh báo học vụ quan trọng. Để nâng cao khả năng nhận diện thị giác, hệ thống ứng dụng các kỹ thuật thiết kế UI/UX hiện đại như làm

nổi bật số buổi vắng mặt bằng sắc đỏ và sử dụng các thẻ nhãn (Badge) màu sắc chuẩn hóa cho cột trạng thái. Sự phối hợp này giúp giảng viên dễ dàng nắm bắt toàn diện tình hình chuyên cần của lớp học và đưa ra các quyết định quản lý chính xác chỉ qua một ánh nhìn tổng quan.

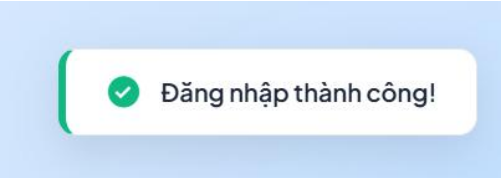
4.4.4. Tối ưu hóa Trải nghiệm người dùng (Advanced UI/UX)

Nhằm đáp ứng các tiêu chuẩn khắt khe của một ứng dụng quản lý hiện đại, hệ thống đặc biệt chú trọng vào việc tối ưu hóa trải nghiệm người dùng (UI/UX) thông qua việc thay thế hoàn toàn các hộp thoại mặc định (alert, confirm) bằng các thành phần giao diện tùy chỉnh không gây nghẽn luồng thực thi (non-blocking). Nổi bật là hệ thống Custom Modal với lớp phủ làm mờ nền và hiệu ứng chuyển động thu phóng (Scale-up), được liên kết chặt chẽ với các hàm gọi lại (Callback) để đảm bảo tính an toàn và chuyên nghiệp khi giảng viên thực hiện điều chỉnh điểm danh thủ công.



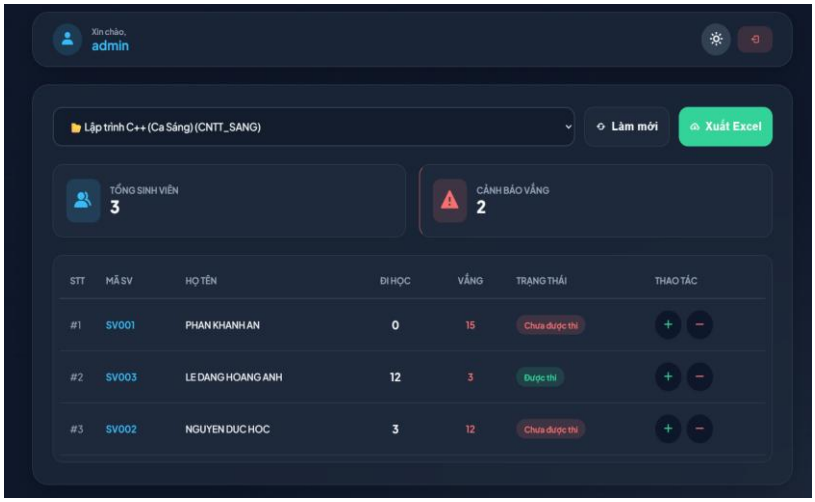
Hình 4.5. Hộp thoại hiện lên khi giảng viên muốn điểm danh bù cho sinh viên

Bên cạnh đó, hệ thống Toast Notification được triển khai để phản hồi tức thì các sự kiện như đăng nhập thành công hoặc cảnh báo lỗi dưới dạng các khung thông báo nhỏ tự động trượt vào góc màn hình và biến mất sau 3 giây, giúp duy trì mạch làm việc thông suốt cho người dùng.



Hình 4.6. Hộp thoại hiện lên khi đăng nhập thành công

Để nâng cao tính công thái học, hệ thống tích hợp chế độ nền tối (Dark Mode) vận hành linh hoạt dựa trên việc điều khiển thuộc tính data-theme và hệ thống biến CSS, cho phép chuyển đổi giao diện tức thì mà không cần tải lại trang. Mọi thiết lập cá nhân hóa này đều được đồng bộ hóa vào localStorage, giúp duy trì không gian làm việc tối ưu cho các phiên truy cập kế tiếp, từ đó tạo nên một môi trường quản trị học vụ hiện đại, thân thiện và hiệu quả.



Hình 4.7. Giao diện Dark Mode

4.4.5. Cơ chế tương tác API và Cập nhật dữ liệu thời gian thực

Hệ thống vận hành theo mô hình SPA, giúp giảng viên cập nhật dữ liệu mượt mà không cần tải lại toàn bộ trang. Thông qua giao diện fetch(), các truy vấn bất đồng bộ được thực hiện liên tục kết hợp với biểu tượng trạng thái xử lý để tối ưu trải nghiệm người dùng. Khi điều chỉnh điểm danh thủ công, gói tin JSON sẽ được gửi lên máy chủ và tự động tái đồng bộ hóa số liệu ngay lập tức. Đồng thời, tính năng xuất báo cáo Excel được xử lý qua

dữ liệu nhị phân (blob), cho phép tải tệp về thiết bị cục bộ một cách nhanh chóng và an toàn.

4.5. Thiết lập thông số mô phỏng

Sau khi hoàn tất kiến trúc, hệ thống được chuyển sang giai đoạn mô phỏng trên môi trường Wokwi nhằm đánh giá hiệu năng trước khi triển khai thực tế. Quá trình này giả lập toàn diện vi điều khiển ESP32 và module RFID-RC522, kết nối trực tiếp đến máy chủ FastAPI và cơ sở dữ liệu Supabase thông qua hạ tầng mạng Wokwi-GUEST. Để đảm bảo tiêu chuẩn đo lường, cấu hình mô phỏng được thiết lập với các tham số kỹ thuật cố định: thời gian vận hành liên tục 60 phút để kiểm chứng độ ổn định của FreeRTOS; tần số lấy mẫu RFID đạt 10Hz (chu kỳ 100ms) giúp bắt tín hiệu tức thời; và thời gian chống dội (Debounce) ở mức 2000ms để ngăn chặn ghi nhận thẻ trùng lặp. Đồng thời, chu kỳ đồng bộ danh sách sinh viên được duy trì 30 phút một lần nhằm cân bằng giữa tính cập nhật và bài toán tối ưu hóa băng thông mạng.

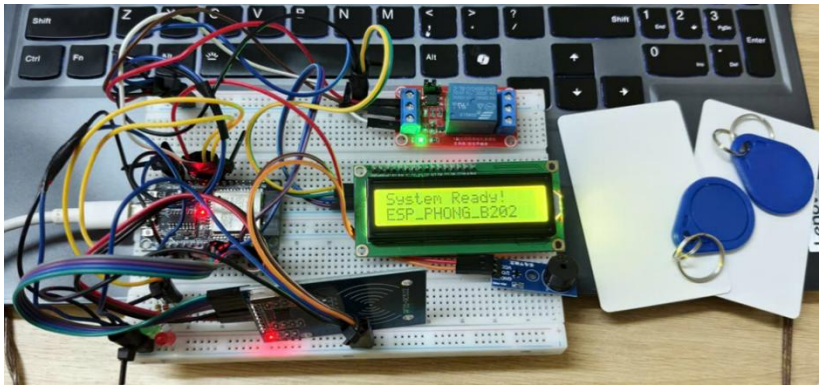
4.6. Thiết kế các kịch bản mô phỏng

Quy trình thực nghiệm được thiết kế xoay quanh ba kịch bản trọng tâm nhằm đánh giá toàn diện năng lực vận hành của hệ thống từ điều kiện tiêu chuẩn đến khắc nghiệt. Đầu tiên, kịch bản đánh giá độ trễ toàn trình (Latency Simulation) được thực hiện qua 10 chu kỳ quét thẻ để đo lường song song tốc độ xử lý tại thiết bị biên và thời gian đồng bộ hóa lên đám mây, giúp xác định tính ổn định thực tế của luồng dữ liệu. Tiếp nối là kịch bản kiểm thử khả năng chịu tải (Stress Simulation) giả lập tình huống ùn tắc với tần suất quẹt 20 thẻ trong 15 giây, từ đó kiểm chứng sự phối hợp nhịp nhàng giữa hàng đợi Queue trên ESP32 và thuật toán Mutex Lock tại máy chủ để bảo vệ tính toàn vẹn dữ liệu. Cuối cùng, kịch bản vận hành ngoại tuyến (Offline Mode Simulation) tập trung xác minh năng lực đối soát độc lập trên bộ nhớ SRAM nội bộ thông qua kỹ thuật Local Caching, đảm bảo hệ thống vẫn ra quyết định chính xác và duy trì lưu thông thông suốt ngay cả khi gặp sự cố đứt gãy kết nối mạng.

CHƯƠNG 5 THIẾT LẬP VÀ TRIỂN KHAI PHẦN CỨNG

5.1. Chuẩn bị linh kiện và thiết lập môi trường phát triển

Để hiện thực hóa hệ thống điểm danh IoT, nhóm sinh viên đã tiến hành khảo sát và lựa chọn các linh kiện điện tử có tính ổn định cao, đáp ứng đúng các tiêu chí thiết kế đã đề ra. Thành phần cốt lõi điều khiển toàn bộ hệ thống là bo mạch phát triển ESP32 Dev Module được tích hợp sẵn khả năng kết nối mạng Wifi. Đi kèm với vi điều khiển là mô-đun đầu đọc thẻ RFID RC522 hoạt động ở tần số 13.56MHz, màn hình LCD 16x2 sử dụng mạch chuyển đổi giao tiếp I2C để tối ưu hóa số lượng chân kết nối, cùng với các cơ cấu chấp hành bao gồm mô-đun rơ-le 5V, đèn LED tín hiệu và mô-đun còi báo động (Active Buzzer) loại 3 chân có tích hợp sẵn mạch đệm nhằm bảo vệ an toàn cho vi điều khiển.



Hình 5.1. Hình ảnh sản phẩm khi sẵn sàng quét

Trong giai đoạn đầu của việc chuẩn bị phần cứng, nhóm đã gặp phải một sự cố kỹ thuật ngoài ý muốn khi bo mạch ESP32 đầu tiên bị hỏng vi mạch (IC) điều khiển nguồn. Trạng thái lỗi này khiến thiết bị không thể giao tiếp với máy tính để nạp mã nguồn, mặc dù các linh kiện ngoại vi khác vẫn hoạt động bình thường. Để đảm bảo tiến độ thực hiện đồ án, nhóm đã nhanh chóng tiến hành mua và thay thế bằng một bo mạch ESP32 mới, giúp cấp nguồn và duy trì tính ổn định cho toàn bộ hệ thống. Về mặt phần mềm, quá trình thiết lập môi trường lập trình trên Arduino IDE diễn ra thuận lợi. Nhóm đã tiến hành cài đặt gói hỗ trợ phần cứng esp32 by Espressif Systems để trình biên dịch có thể hoạt động với kiến trúc lõi kép của ESP32. Song song đó, các thư viện quan trọng cũng được tích hợp vào dự án thông qua Library Manager, bao gồm LiquidCrystal_I2C dùng để điều khiển màn hình,

MFRC522 phục vụ cho việc giao tiếp với đầu đọc thẻ qua chuẩn SPI, cùng với các thư viện ArduinoJson, HTTPClient và WiFi đóng vai trò quản lý kết nối mạng và xử lý các gói tin JSON khi giao tiếp với giao thức RESTful API của máy chủ.

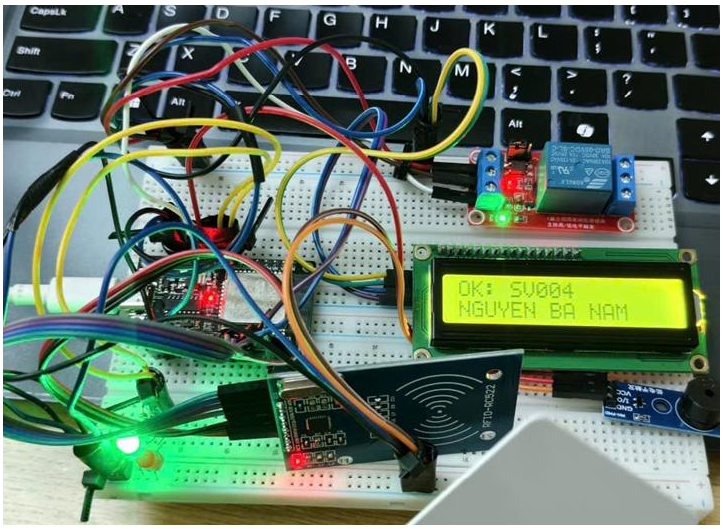
5.2. Lắp ráp và kiểm thử theo từng phân hệ (Incremental Testing)

Thay vì tiến hành lắp ráp hoàn chỉnh và nạp mã nguồn tổng hợp cho toàn bộ hệ thống ngay từ đầu, nhóm đã áp dụng chiến lược kiểm thử tăng dần (Incremental Testing). Phương pháp luận này giúp nhóm dễ dàng cô lập lỗi, kiểm tra độ chính xác của từng linh kiện trước khi tiến hành ghép nối phức tạp. Quá trình kiểm thử bắt đầu với các thiết bị đầu ra cơ bản. Đối với màn hình LCD, nhóm sử dụng giao tiếp I2C thông qua hai chân SDA và SCL (lần lượt tương ứng với chân D21 và D22 trên ESP32). Hệ thống đèn LED, rơ-le và còi báo động cũng được kết nối và chạy thử nghiệm. Điểm khác biệt quan trọng so với môi trường mô phỏng trên Wokwi là việc sử dụng mô-đun còi báo động chủ động (Active Buzzer) 3 chân. Thiết bị này yêu cầu kết nối đầy đủ các chân VCC, GND và chân tín hiệu (I/O) thay vì loại 2 chân cơ bản, giúp âm thanh cảnh báo phát ra ổn định và không gây sụt áp cho bo mạch chủ. Tiếp theo, nhóm tiến hành kiểm thử mô-đun đọc thẻ RFID RC522. Trong khi hệ thống mô phỏng Wokwi bị giới hạn và phải sử dụng chuẩn giao tiếp UART, mạch thực tế yêu cầu sử dụng chuẩn giao tiếp đồng bộ tốc độ cao SPI (Serial Peripheral Interface). Khi tiến hành kết nối phần cứng và nạp mã nguồn đọc mã định danh (UID) của thẻ, nhóm đã gặp hiện tượng cổng nối tiếp (Serial Monitor) in ra các ký tự rác không xác định. Qua quá trình phân tích chẩn đoán, nguyên nhân được xác định là do sự mất đồng bộ về tốc độ truyền tải (Baud rate). Mã nguồn của ESP32 được thiết lập phát tín hiệu ở tốc độ 115200 baud, trong khi phần mềm Arduino IDE đang tiếp nhận ở mức mặc định là 9600 baud. Sau khi điều chỉnh và đồng bộ lại tốc độ truyền trên phần mềm, hệ thống đã giao tiếp thành công, in ra chính xác phiên bản Firmware của vi mạch RC522 và trích xuất thành công mã UID khi có thẻ tiến vào vùng từ trường.

5.3. Tích hợp hệ thống và xử lý sự cố vận hành thực tế

Sau khi hoàn tất giai đoạn kiểm thử đơn lẻ, nhóm tiến hành ghép nối toàn bộ sơ đồ mạch điện và nạp mã nguồn điều khiển tổng hợp (Firmware). Phiên bản mã nguồn này

không chỉ kết hợp các luồng giao tiếp phần cứng mà còn ứng dụng hệ điều hành thời gian thực (FreeRTOS) nhằm gán tác vụ truyền tải mạng ngầm (TaskUpload) chạy độc lập trên lõi số 0 (Core 0), giúp thao tác quẹt thẻ của người dùng không bị gián đoạn.



Hình 5.2. Màn hình sản phẩm hiển thị thông báo khi quét thành công

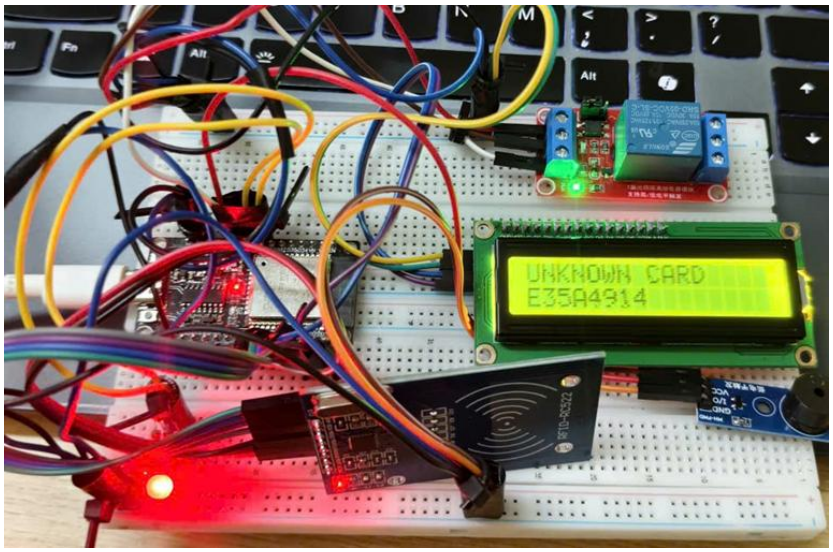
Tuy nhiên, khi đưa vào vận hành thực tế tích hợp cùng hệ thống máy chủ (Backend FastAPI), hệ thống đã phát sinh hai vấn đề logic nghiệp vụ và đã được nhóm đưa ra các giải pháp xử lý triệt để.

5.3.1. Xử lý lỗi đồng bộ thời gian (Lỗi 400 Bad Request)

Hiện tượng đầu tiên là thiết bị nhận diện thẻ hợp lệ nhưng hệ thống máy chủ lại từ chối giao dịch với mã lỗi 400 Bad Request cùng thông báo không tìm thấy lịch học. Qua quá trình gỡ lỗi (debug) bằng cách in phản hồi từ máy chủ ra màn hình giám sát, nhóm phát hiện nguyên nhân xuất phát từ việc vi điều khiển ESP32 lấy mốc thời gian thực hiện điểm danh gửi lên máy chủ là 0 (tương đương với mốc thời gian Epoch năm 1970). Điều này xảy ra do thiết bị thao tác quét thẻ trước khi hoàn tất việc đồng bộ đồng hồ mạng (NTP) qua Wifi. Giải pháp được áp dụng là tái cấu trúc hàm khởi tạo setup(), bổ sung một vòng lặp kiểm tra có điều kiện nhằm ép buộc vi điều khiển ESP32 phải chờ cho đến khi lấy được thời gian chuẩn từ mạng Internet (mốc Epoch Time phản ánh đúng thời gian hiện tại) trước khi chuyển sang trạng thái sẵn sàng đón nhận thẻ.

5.3.2. Xử lý lỗi vi phạm toàn vẹn dữ liệu

Trong quá trình thực nghiệm, hệ thống đã phát sinh lỗi 500 Internal Server Error khi thực hiện quét các thẻ RFID chưa được đăng ký. Nguyên nhân kỹ thuật được xác định là do ràng buộc Khóa ngoại (Foreign Key) trong cơ sở dữ liệu Supabase: bảng nhat_ky_diem_danh yêu cầu mọi mã số sinh viên phải tồn tại trong bảng sinh_vien để đảm bảo tính toàn vẹn. Để khắc phục triệt để vấn đề này mà không làm phá vỡ cấu trúc thiết kế, nhóm đã triển khai giải pháp "Sinh viên ảo". Một bản ghi đặc biệt được bổ sung vào bảng sinh_vien với mã số "UNKNOWN" và tên định danh là "Khách lạ / Thẻ chưa đăng ký". Nhờ kiến trúc này, khi có thẻ lạ tiếp cận, hệ thống không chỉ bật đèn đỏ và còi báo mà còn cho phép máy chủ ghi nhận thành công sự kiện vào nhật ký.



Hình 5.3. Màn hình sản phẩm hiển thị thông báo khi quét thất bại

Việc áp dụng giải pháp "Sinh viên ảo" mang lại giá trị kép cho hệ thống: vừa bảo vệ tính nhất quán của cơ sở dữ liệu theo tiêu chuẩn quan hệ, vừa nâng cao năng lực giám sát an ninh. Thay vì bỏ qua các lỗi truy cập, giờ đây mọi thông tin về thời gian và ID thẻ lạ đều được lưu vết chi tiết, tạo nguồn dữ liệu quan trọng để người quản lý phân tích các hành vi xâm nhập hoặc hỗ trợ kịp thời sinh viên gặp sự cố về thẻ. Đây chính là nền tảng vững chắc để hệ thống duy trì sự ổn định và sẵn sàng mở rộng các tính năng thống kê, báo cáo chuyên sâu trong tương lai.

CHƯƠNG 6 KẾT QUẢ, ĐÁNH GIÁ MÔ PHỎNG VÀ NGOÀI THỰC NGHIỆM

6.1. Kết quả, đánh giá mô phỏng

6.1.1. Kịch bản 1: Đánh giá tốc độ phản hồi của hệ thống (Latency)

Kịch bản này đo lường song song Độ trễ cục bộ (Local Latency) và Độ trễ đám mây (Cloud Latency) trong điều kiện mạng tiêu chuẩn. Quy trình sử dụng ESP32 kết nối FastAPI/Supabase qua HTTP/REST, thực hiện 10 chu kỳ quét thẻ với khoảng nghỉ 5 giây. Kết quả ghi nhận thời gian từ lúc cảm biến nhận tín hiệu đến khi phần cứng phản hồi và dữ liệu được xác nhận lưu trữ thành công trên máy chủ.

Bảng 6.1. Số liệu tốc độ phản hồi phần cứng ở môi trường mô phỏng

Lần thử	Độ trễ Local (LCD/Còi)	Độ trễ Cloud (Database Update)
1	0.7	3.9
2	0.9	3.8
3	0.8	3.8
4	0.7	3.7
5	0.8	3.8
6	0.9	3.9
7	0.8	4.1
8	0.9	4.0
9	0.9	3.4
10	0.7	3.8
Trung bình	0.8	3.8

Nhận xét: Kết quả trung bình 0.8 giây (nội bộ) và 3.8 giây (đám mây) khẳng định hiệu quả vượt trội của kiến trúc Smart Edge Caching. Việc đối soát trực tiếp trên RAM giúp thiết bị phản hồi âm thanh và hiển thị tên sinh viên gần như tức thời. Mặc dù tiến độ đồng bộ lên Cloud phụ thuộc vào hạ tầng mạng nhưng vẫn đảm bảo tính thời gian thực cho Web

Dashboard. Sự ổn định giữa các lần thử cho thấy hệ thống vận hành bền bỉ, không xảy ra hiện tượng nghẽn tác vụ cục bộ.

6.1.2. Kịch bản 2: Đánh giá độ ổn định khi chịu tải cao

Kịch bản này kiểm chứng sức chịu đựng của hệ thống trong tình huống ùn tắc thông qua việc đánh giá cấu trúc hàng đợi (Queue) trên ESP32 và khả năng bảo toàn dữ liệu của khóa Mutex trên FastAPI. Quy trình thực nghiệm tiến hành tăng dần áp lực tải từ 5 thẻ/3 giây đến mức cực hạn 20 thẻ/15 giây để xác định tỷ lệ thất thoát gói tin.

Bảng 6.2. Số liệu đánh giá độ ổn định trong môi trường mô phỏng

Tốc độ quét	Số thẻ quét	Số bản ghi trên database	Tỉ lệ rớt gói tin
5 thẻ / 3 giây	5	5	0
5 thẻ/ 5 giây	5	5	0
10 thẻ / 10 giây	10	10	0
15 thẻ / 10 giây	15	15	0
20 thẻ / 15 giây	20	20	0

Nhận xét: Kết quả thực nghiệm cho thấy hệ thống vận hành ổn định với tỷ lệ rớt gói tin bằng 0% ở mọi cấp độ tải. Thành công này minh chứng cho tính hiệu quả của cấu trúc xQueue trong FreeRTOS khi tách biệt hoàn toàn tác vụ quét thẻ và truyền tải mạng giữa hai lõi xử lý, ngăn chặn tình trạng nghẽn lệnh tại thiết bị biên. Đồng thời, sự kết hợp giữa xử lý bất đồng bộ và khóa Mutex tại Backend đã loại bỏ hoàn toàn rủi ro Race Condition, đảm bảo mọi dữ liệu điểm danh đều được ghi nhận tuần tự và chính xác tuyệt đối, đáp ứng tốt nhu cầu thực tế tại các giảng đường có sĩ số lớn.

6.1.3. Kịch bản 3: Đánh hệ thống trong trường hợp ngoại tuyến

Kịch bản này kiểm tra tính tự chủ của thiết bị khi mất kết nối đột ngột nhằm đảm bảo dịch vụ liên tục. Bằng cách ngắt Internet giả lập trên Wokwi, nhóm xác minh khả năng đối soát UID dựa trên dữ liệu nạp sẵn vào SRAM. Cơ chế chuyển đổi tức thì sang Local

Caching giúp hệ thống phản hồi mượt mà, đảm bảo vận hành ổn định mà không phụ thuộc vào hạ tầng mạng thực tế.

Bảng 6.3. Số liệu khi kiểm tra hệ thống ở các điều kiện khác nhau trong môi trường mô phỏng

Trạng thái kết nối	Tốc độ đối soát (ms)	Khả năng phản hồi ngoại vi	Trạng thái dữ liệu trong Queue
Trực tuyến (Online)	< 50ms	Tức thời	Đã đẩy lên Server thành công
Ngoại tuyến (Offline)	< 50ms	Tức thời	Lưu trữ tạm thời, chờ kết nối
Phục hồi kết nối	< 50ms	Tức thời	Tự động đồng bộ các gói tin tồn đọng

Nhận xét: Kết quả cho thấy hệ thống vận hành bền bỉ với tốc độ phản hồi nội bộ ổn định (< 50ms). Khi mất mạng, cấu trúc xQueue đóng vai trò vùng đệm an toàn, lưu giữ mọi bản tin điểm danh và tự động đồng bộ ngay khi có kết nối trở lại. Sự kết hợp giữa xử lý tại biên (Edge Computing) và điện toán đám mây giúp bảo toàn dữ liệu tuyệt đối và không làm gián đoạn luồng di chuyển của sinh viên trong mọi tình huống hạ tầng.

6.2. Kết quả, đánh giá ngoài thực nghiệm

6.2.1. Kịch bản 1: Đánh giá tốc độ phản hồi của hệ thống (Latency)

Giai đoạn thực nghiệm thực tế được triển khai nhằm kiểm chứng hiệu năng hệ thống trong môi trường vật lý. Kịch bản này tập trung đo lường độ trễ phản hồi tại thiết bị và thời gian đồng bộ dữ liệu lên máy chủ dưới tác động trực tiếp của hạ tầng mạng trong môi trường thực tế.

Bảng 6.4. Số liệu tốc độ phản hồi phần cứng ở môi trường thực tế

Lần thử	Độ trễ Local (LCD/Còi)	Độ trễ Cloud (Database Update)
1	0,6	4,6
2	0,7	4,7
3	0,7	4,1
4	0,6	4,4
5	0,6	4,9
6	0,4	4,1
7	0,6	4,9
8	0,5	4,4
9	0,4	4,6
10	0,5	4,6
Trung bình	0,61	4.53

Nhận xét: Kết quả cho thấy độ trễ cục bộ trung bình (0.61 giây) nhanh hơn so với mô phỏng, khẳng định hiệu năng xử lý mạnh mẽ của chip ESP32 thực tế trong việc điều khiển ngoại vi. Dù độ trễ Cloud tăng lên 4.53 giây do ảnh hưởng từ nhiễu sóng Wifi và khoảng cách địa lý đến máy chủ, kiến trúc đa nhân vẫn đảm bảo phản hồi tại chỗ (âm thanh, màn hình) diễn ra tức thì. Điều này minh chứng hệ thống hoàn toàn đáp ứng tốt trải nghiệm người dùng tại hiện trường mà không bị phụ thuộc vào tốc độ đường truyền Internet.

6.2.2. Kịch bản 2: Đánh giá độ ổn định khi chịu tải cao

Kịch bản này kiểm tra khả năng đáp ứng liên tục của hệ thống trong giờ cao điểm. Do quy trình phần cứng yêu cầu đèn LED và còi Buzzer phải hoàn tất chu kỳ báo hiệu trước khi chấp nhận lượt mới, tốc độ thực nghiệm được duy trì ở mức 1 giây/thẻ. Đây là ngưỡng tương tác thực tế giúp sinh viên nhận biết trạng thái điểm danh và tránh xung đột phần mềm.

Bảng 6.5. Số liệu đánh giá độ ổn định trong môi trường thực tế

Đợt thử nghiệm	Số lượng thẻ quét	Tổng thời gian thực hiện	Số bản ghi thành công	Tỷ lệ chính xác
Đợt 1	5 thẻ	5 giây	5/5	100%
Đợt 2	10 thẻ	10 giây	10/10	100%
Đợt 3	20 thẻ	20 giây	20/20	100%
Đợt 4	30 thẻ	30 giây	30/30	100%

Nhận xét: Hệ thống đạt tỷ lệ chính xác tuyệt đối 100% ở mọi cấp độ tải. Việc ứng dụng cấu trúc xQueue giúp ESP32 tiếp nhận UID tức thì và đẩy vào hàng đợi để xử lý ngầm, giải phóng tài nguyên CPU ngay lập tức để sẵn sàng cho thẻ tiếp theo. Kết quả thực nghiệm khẳng định hệ thống hoàn toàn loại bỏ được tình trạng nghẽn lệnh, đảm bảo lưu thông thông suốt và tin cậy tại cửa lớp ngay cả trong khung giờ cao điểm..

6.2.3. Kịch bản 3: Đánh giá độ ổn định của hệ thống khi vận hành khi ngoại tuyến

Kịch bản này kiểm chứng khả năng duy trì dịch vụ liên tục (Business Continuity) của thiết bị khi hạ tầng Wifi tại trường học gặp sự cố. Quy trình thực hiện bằng cách ngắt kết nối bộ phát Wifi, tiến hành quét 15 thẻ sinh viên đã lưu trong RAM (Local Caching) và quan sát khả năng tự động đồng bộ sau khi khôi phục mạng.

Bảng 6.6. Số liệu kiểm tra hệ thống ở các điều kiện khác nhau trong môi trường thực tế

Trạng thái	Số thẻ quét	Phản hồi phần cứng (LED/LCD)	Trạng thái đồng bộ
Mất mạng (Offline)	15	Hoạt động bình thường (Dưới 0.1s)	Lưu tạm vào RAM (Queue)
Có mạng lại (Online)	0	Không bị gián đoạn	Tự động đẩy 15 bản ghi lên Cloud

CHƯƠNG 7 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trải qua quá trình nghiên cứu, thiết kế và thi công, nhóm đã hoàn thành mục tiêu tổng quát là xây dựng thành công một hệ thống IoT điểm danh sinh viên hoạt động theo kiến trúc Edge-to-Cloud phân tán. Về mặt phần cứng, thiết bị đầu cuối sử dụng vi điều khiển ESP32 và module RFID RC522 đã được tối ưu hóa sâu để đạt tốc độ phản hồi tại chỗ trung bình chỉ 0,8 giây. Thành công lớn nhất về mặt kỹ thuật là việc ứng dụng thành công kỹ thuật Smart Edge Caching, giúp thiết bị tự động đồng bộ danh sách sinh viên về RAM nội bộ, từ đó duy trì khả năng ra quyết định chính xác ngay cả khi điều kiện mạng Internet tại phòng học không ổn định. Đối với hệ thống phần mềm, nhóm đã làm chủ hoàn toàn quy trình xây dựng máy chủ Backend bằng framework FastAPI và cơ sở dữ liệu Supabase đạt chuẩn hóa 3NF. Việc tích hợp cơ chế khóa tương hỗ Mutex Lock đã giải quyết triệt để bài toán xung đột luồng khi xử lý dữ liệu đồng thời, minh chứng qua kết quả kiểm thử chịu tải với tỉ lệ rớt gói tin bằng 0% trong các tình huống cao điểm. Bên cạnh đó, giao diện Web quản trị được xây dựng theo kiến trúc SPA hiện đại, cung cấp cho giảng viên bộ công cụ quản lý toàn diện từ giám sát sĩ số tức thời đến khả năng trích xuất báo cáo chuyên cần ra định dạng Excel một cách tự động, minh bạch và chuyên nghiệp. Dữ liệu thực nghiệm thu được cho thấy hệ thống không chỉ đạt độ tin cậy cao về mặt kỹ thuật mà còn mang lại giá trị thực tiễn lớn trong việc hiện đại hóa quản trị giáo dục, giúp tiết kiệm thời gian và tối ưu hóa quy trình vận hành tại nhà trường. Mặc dù đã đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số hạn chế nhất định như sự phụ thuộc vào tính ổn định của nguồn điện và chưa tích hợp các biện pháp chống gian lận vật lý chuyên sâu.

Trong tương lai, nhóm đề xuất hướng phát triển tiếp theo là tích hợp thêm AI nhận diện khuôn mặt để đối chiếu song song với thẻ RFID, nhằm loại bỏ hoàn toàn tình trạng điểm danh hộ. Bên cạnh đó, việc phát triển ứng dụng di động cho sinh viên để tra cứu lịch sử chuyên cần và chuyển đổi sang thiết kế mạch in PCB chuyên dụng sẽ giúp hệ thống đạt độ bền công nghiệp, sẵn sàng cho việc triển khai thực tế trên quy mô toàn trường, hướng tới xây dựng một hệ sinh thái học đường thông minh và toàn diện.

TÀI LIỆU THAM KHẢO

[1] B. K. h. v. C. nghệ, "Xu hướng công nghệ Internet of Things (IoT) và ứng dụng trong xây dựng Đô thị thông minh," *Tạp chí Khoa học và Công nghệ Việt Nam,* 2019.

[2] Dientutuonglai. "ESP32 là gì?" https://dientutuonglai.com/esp32-la-gi.html (accessed 01 April 2026.

[3] Espressif Systems. "ESP32 Technical Reference Manual (Version 5.7)." https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf (accessed 01 April 2026.

[4] Dientutuonglai. "Sơ đồ chân ESP32." https://dientutuonglai.com/so-do-chan-esp32.html (accessed 01 April 2026.

[5] IoTZone. "Giới thiệu ESP32 là gì?" https://iotzone.vn/esp32/esp32-co-ban/gioi-thieu-esp32-la-gi/ (accessed 01 April 2026.

[6] Real Time Engineers Ltd, "FreeRTOS Reference Manual V8.2.1," Real Time Engineers Ltd, 2015. Accessed: 01 April 2026.

[7] Trackify. "Công nghệ RFID là gì? Nguyên lý hoạt động – Cấu tạo – Ứng dụng." https://trackify.vn/rfid-la-gi/ (accessed 01 April 2026.

[8] ViettelIDC. "RFID là gì? Tìm hiểu công nghệ nhận dạng qua sóng vô tuyến." https://viettelidc.com.vn/tin-tuc/rfid-la-gi-tim-hieu-cong-nghe-nhan-dang-qua-song-vo-tuyen-4837 (accessed 01 April 2026.

[9] Tinhte.vn. "Thẻ Mifare 1K 13.56MHz là gì? Ứng dụng thực tế chi tiết." https://tinhte.vn/thread/the-mifare-1k-13-56mhz-la-gi-ung-dung-thuc-te-chi-tiet.4062724/ (accessed 01 April 2026.

[10] NXP Semiconductors, "MFRC522: Standard performance MIFARE and NTAG frontend - Data Sheet," NXP Semiconductors, Rev. 3.9, 2026. Accessed: 01 April 2026. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

[11] Studocu. "Giao thức SPI: Cấu trúc, hoạt động và ứng dụng trong điện tử." https://www.studocu.vn/vn/document/dai-hoc-kinh-te-quoc-dan/quy-hoach-va-bao-ton-tai-nguyen/giao-thuc-spi-cau-truc-hoat-dong-va-ung-dung-trong-dien-tu/129125583 (accessed 01 April 2026.

[12] IoTZone. "Tất tần tật về giao thức I2C bạn cần biết." https://iotzone.vn/blog/giao-thuc-i2c-la-gi/ (accessed 1 April 2026.

[13] B. Blanchon. "ArduinoJson 7 Documentation: Efficient JSON serialization for embedded C++." https://arduinojson.org/ (accessed 01 April 2026.

[14] Supabase Inc. "Supabase Architecture and PostgreSQL Documentation." https://supabase.com/docs (accessed 01 April 2026.

[15] The PostgreSQL Global Development Group. "PostgreSQL 18 Documentation." https://www.postgresql.org/docs/ (accessed 01 April 2026.

[16] S. Ramírez. "FastAPI Documentation: High performance, easy to learn, fast to code, ready for production." https://fastapi.tiangolo.com/ (accessed 01 April 2026.

PHỤ LỤC

Mô phỏng thiết bị biên trên Wokwi: https://wokwi.com/projects/454856473504871425

Mã nguồn cloud host web và backend: https://huggingface.co/spaces/danganhle0623/IoT

Toàn bộ mã nguồn của dự án: https://github.com/anphan991/IoT-Student-Attendance-System